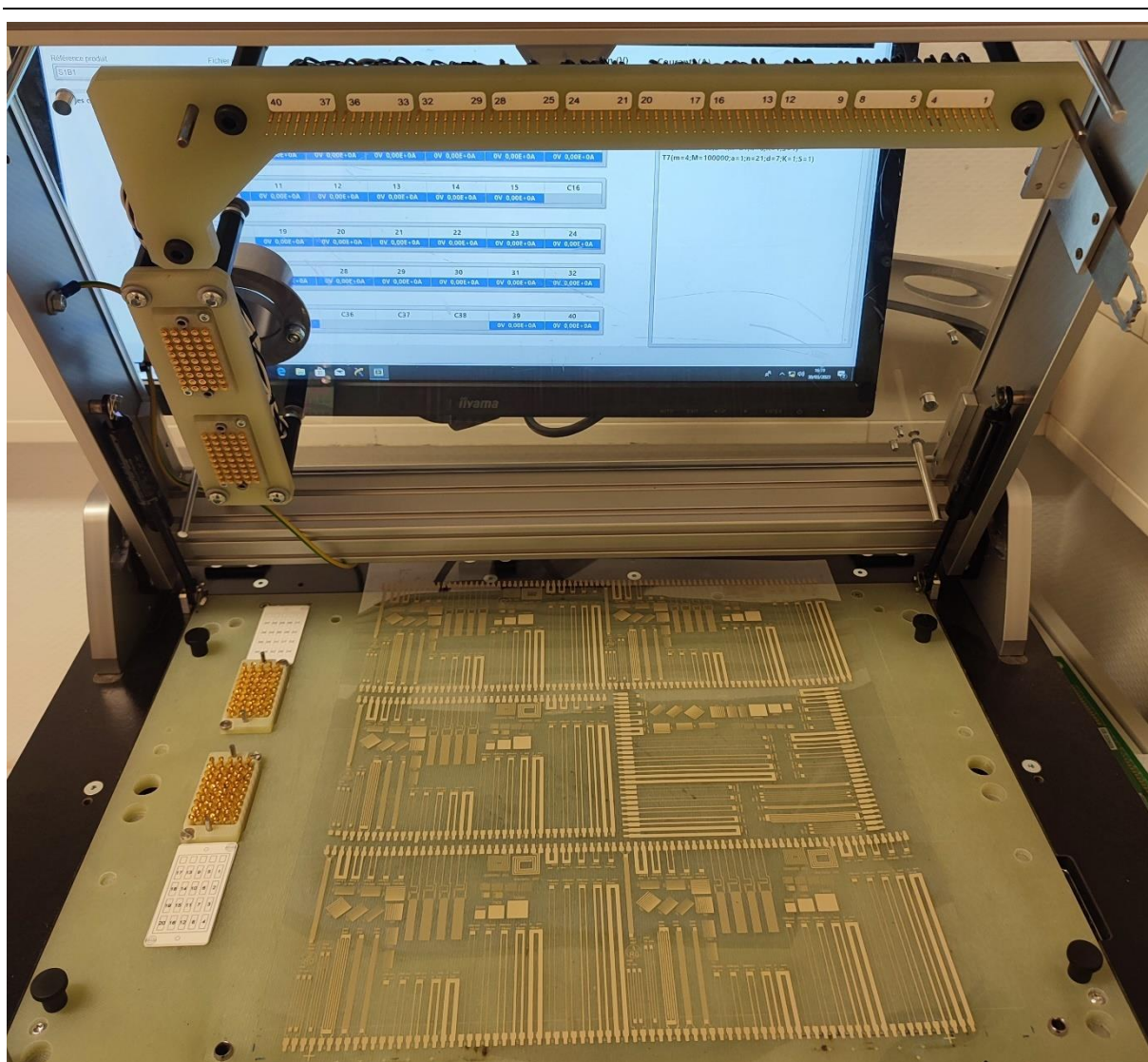


Étude et mise en place d'un protocole de caractérisation électrique d'encres conductrices réalisées par sérigraphie

Rapport de stage BUT 2

Léo QUESNOIT

2022-2023



Maître de stage : Alexandre PEREIRA

Tutrice de stage : Sandrine BEAUQUIS

Je voudrais tout d'abord remercier Richard LAUCOURNET, chef du Département des Technologies des Nouveaux Matériaux et Audrey MARTINENT, chef du Service des Composants et Surfaces Fonctionnalisées de m'avoir accueilli au sein de leurs département et service.

Je souhaiterais également remercier Didier GALLAIRE, responsable du Laboratoire des Composants Hétérogènes de m'avoir permis de travailler dans un environnement de qualité.

J'adresse ma gratitude à Alexandre PEREIRA, mon maître de stage, pour m'avoir accompagné tout au long de mon stage, pour avoir répondu à toutes mes questions mais aussi pour tous ses précieux conseils.

Je remercie également Sébastien NOEL et Jamal TALLAL pour avoir pris le temps de m'avoir formé et accompagné sur les différents appareils dont j'avais le besoin. Je remercie également Romain REYES qui m'a aidé à résoudre les conflits de programmation.

De plus, je remercie les thésards et stagiaire qui ont contribué à m'aider tout au long de mon stage et qui ont contribué à ce qu'il se déroule dans les meilleures conditions.

Enfin, je souhaiterais remercier Sandrine BEAUQUIS, ma tutrice de stage, pour s'être déplacée afin de visiter les locaux du CEA et d'avoir répondu à mes questionnements avec rapidité.

Table des matières

1	Introduction	7
2	Présentation de l'environnement de l'étude.....	7
2.1	Présentation du CEA	7
2.1.1	Histoire du CEA	7
2.1.2	Le CEA de Grenoble	8
3	Cahier des charges.....	8
3.1	Les Objectifs.....	8
3.2	Méthodologie.....	9
3.2.1	Technique d'impression par sérigraphie.....	9
3.2.2	Les pistes conductrices	10
3.2.3	La profilométrie mécanique	11
3.2.4	Banc de mesures électriques.....	12
4	Mesures expérimentales.....	13
4.1	Mesures profilométriques.....	13
4.2	Mesures DACTEM.....	18
4.3	Estimation de la résistivité.....	20
5	Programmation.....	23
5.1	Les attentes du programme.....	23
5.2	Développement.....	23
5.2.1	Concaténation de fichiers de données	24
5.2.2	Programme de caractérisation	26
6	Conclusion.....	28
7	Bibliographie.....	30
8	Annexes.....	31

1 Introduction

Dans une société en pleine révolution technologique, l'écologie et l'économie poussent la technologie à évoluer avec de nouveaux enjeux. C'est par exemple le cas pour les des circuits imprimés. En effet, les circuits imprimés qu'il est possible de trouver principalement dans les ordinateurs, calculatrice et appareil électroménager sont les Printed Circuit Boards (PCB). Un circuit imprimé est un substrat (généralement en résine époxy renforcée de fibres de verre : FR-4) permettant de lier électriquement des composants électroniques et ainsi réaliser un circuit électronique ayant une fonction donnée (pilotage d'un appareil électronique, traitement du signal...). Il est classiquement constitué d'une alternance de couches de cuivre et de couches isolantes électriquement. Les couches de cuivre sont gravées (gravure chimique, mécanique ou laser) au travers d'un masque comportant les motifs finaux des pistes conductrices et des empreintes des composants. Ces pistes relient ainsi différentes empreintes devant accueillir les composants électroniques qui seront reportés ultérieurement. Ces circuits sont difficilement recyclables et leur procédé de fabrication est énergivore. Pour remplacer cette technologie, une solution alternative consiste à imprimer des encres conductrices sur un substrat souple par sérigraphie.

Les principaux avantages de ce procédé sont :

La flexibilité du substrat, qui permet de lui donner la forme voulue ; le substrat sur lequel l'encre est imprimée peut-être du papier ou du plastique, il est par conséquent beaucoup plus simple de recycler les produits et le procédé s'inscrit dans une démarche d'économie circulaire et enfin le coût est faible.

Néanmoins, avec une résolution minimum typique de 100µm, ce procédé engendre certaines limitations dans la conception des circuits. De plus, la qualité d'impression dépend de l'encre utilisée ainsi que des paramètres de réglage d'impression. En somme, la sérigraphie est un procédé qui présente un bon rapport qualité/énergie/prix s'inscrivant dans une démarche écologique mais n'est pas la technologie qui remplacera les PCB dans tous les domaines.

Récemment, ce type de circuit a été mis en œuvre dans des objets communiquant (antennes RFID), dans les tableaux de bord des voitures, ou encore dans des pansements intelligents.

Deux missions sont retenues pour le cadre du stage, la première est la mise en place d'un protocole de caractérisation électrique d'encres conductrices réalisées par sérigraphie ainsi que la mise en place d'une base de données. La deuxième mission est d'utiliser ce protocole pour étudier une encre en fonction de paramètres morphologiques ainsi que du temps et de la température de recuit. Le but final étant de permettre au personnel du CEA de pouvoir suivre les performances électriques d'encres conductrices dans des conditions de répétabilité optimales.

Dans ce rapport, nous allons voir dans un premier temps l'environnement de travail au sein du CEA et son histoire. Nous dresserons ensuite le cahier des charges, par la suite, nous détaillerons les mesures résistif et morphologique. Nous terminerons par le développement du programme informatique qui permettra de caractériser les encres conductrices.

2 Présentation de l'environnement de l'étude

2.1 Présentation du CEA

2.1.1 Histoire du CEA

A l'origine du premier réacteur nucléaire français, de la caméra infrarouge de James Webb, du laser mégajoule et bien plus encore, le Commissariat à l'Énergie Atomique et aux énergies alternatives (CEA) est un organisme français de recherche en sciences et technologies classé en établissement public à

caractère industriel et commercial (EPIC), spécialisé dans les domaines de l'énergie nucléaire, de la défense, de la recherche technologique pour l'industrie et de la recherche fondamentale.

Créé le 18 octobre 1945 après que le général de Gaulle l'ait ordonné, le CEA est un acteur majeur dans le domaine de la recherche et de l'innovation en France, et est considéré comme l'un des plus importants centres de recherche en Europe. A sa création, le CEA a pour mission de développer les technologies de l'énergie nucléaire pour répondre aux besoins énergétiques de la France, tout en veillant à la sûreté et à la sécurité nucléaires.

Mais le CEA d'aujourd'hui ne se limite pas au nucléaire, de fait, il possède un large portefeuille de technologies et de compétences dans les domaines de la recherche fondamentale, de la défense, de l'énergie nucléaire, des matériaux, de la biologie, de la santé et des technologies de l'information et de la communication. Il travaille en étroite collaboration avec les universités, les entreprises et les organismes internationaux pour développer des technologies innovantes et améliorer la qualité de vie des citoyens.

Le CEA est également impliqué dans des projets de recherche sur l'environnement, la lutte contre le changement climatique et la production d'énergies renouvelables. Il s'efforce de fournir des solutions durables pour répondre aux défis énergétiques actuels et futurs, tout en veillant à ce que ces solutions soient sûres, fiables et respectueuses de l'environnement.

De nos jours, le CEA apparaît comme un expert à l'international dans énormément de domaines et faisant partie du top 100 des entreprises les plus innovantes au monde depuis 10 ans maintenant.

2.1.2 Le CEA de Grenoble

Le CEA de Grenoble est un centre de recherche de pointe spécialisé dans les sciences de la matière et des technologies. Le CEA de Grenoble est un des neuf instituts du CEA national, avec une activité de recherche axée sur les matériaux avancés, la nanotechnologie, la biologie, les sciences de l'environnement et les technologies de l'information.

Fondé en 1956, le CEA de Grenoble est un leader mondial en matière de recherche sur les matériaux avancés, avec un accent particulier sur les matériaux pour l'énergie. Réunissant 4500 collaborateurs, le centre possède une expertise sans égal dans la caractérisation de matériaux à l'échelle nanométrique et le développement de nouvelles méthodes pour la production de matériaux avancés.

Centre d'excellence, il est à l'origine du premier pôle d'excellence européen en micro et nanotechnologies. Il aura aussi un rôle d'incubateur d'entreprise, comme avec l'INES ou ISORG et bien d'autres.

En résumé, le CEA de Grenoble est un centre de recherche exceptionnel qui travaille sur des projets de pointe pour résoudre les défis les plus complexes de notre temps, dans les domaines des matériaux avancés, des technologies de l'information, de la biologie et des sciences de l'environnement. C'est un acteur majeur de la communauté scientifique et un catalyseur pour l'innovation et le développement économique.

3 Cahier des charges

Afin de cerner les attendues du stage, il a été au préalable nécessaire de mettre en place un cahier des charges rappelant tous les objectifs. De plus, ce cahier des charges servira de base pour la construction du programme afin de caractériser des encres conductrices.

3.1 Les Objectifs

La mission principale est **de mettre en place un protocole** permettant de caractériser des encres conductrices imprimées par sérigraphie. Pour ce faire il faudra calculer la résistivité des pistes, elle est définie par l'équation (1) :

$$\rho = R \times \frac{S}{l} \quad (1)$$

Avec : ρ = Résistivité en $\Omega.m$

R = Résistance en Ω

S = Section en m^2

l = Longueur de piste en m

Par conséquent, cela implique de réaliser des mesures morphologiques (de sections et de longueurs différentes) ainsi que de résistance. Enfin, il est important de noter que les propriétés électriques des pistes varient en fonction des paramètres du recuit, il faudra alors aussi étudier la température et le temps de recuit des pistes.

Le but de ce protocole est d'automatiser les différentes mesures électriques et morphologiques afin de fournir au CEA un outil permettant de suivre de façon reproductible les performances des encres. Pour satisfaire cette attente, il faudra mettre au point **un programme permettant de traiter les données et de les stocker directement dans une base de données**, le but étant de sauvegarder toutes les mesures effectuées.

De plus, ce même programme sera utilisé pour **réaliser l'étude de l'encre conductrice NEWLONG BECTRON**.

3.2 Méthodologie

3.2.1 Technique d'impression par sérigraphie

La sérigraphie est une technique bien connue pour l'impression d'encre conductrice, en effet, cette technique présente l'avantage d'être rapide, peu onéreuse et permet d'imprimer des pistes sur des substrats souples. Ces encres sont composées de particules de métal (très souvent de l'argent) et d'une matrice de polymère. Ces particules de métal sont en suspension dans la matrice ayant des propriétés conductrices. Le principe de la technique de sérigraphie est le suivant :

Un masque est d'abord imaginé et développé. Ce dernier se présente sous la forme d'un pochoir composé de fils entrelacés formant un maillage. Ce masque est conçu pour laisser passer l'encre conductrice dans certaines zones, afin de représenter un motif. La figure 1 ci-dessous représente une image d'un masque de sérigraphie réalisée par Microscopie Electronique à Balayage (MEB).

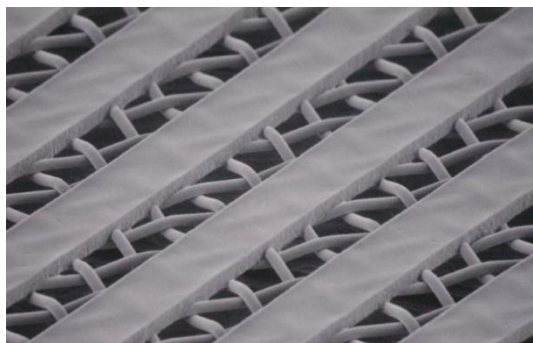


FIGURE 1 : PHOTO D'UN MASQUE DE SERIGRAPHIE

Le masque est placé au-dessus du substrat choisi à une distance prédéfinie par l'équipementier. Une racle en téflon est utilisée pour étaler l'encre sur le masque et ainsi former les motifs souhaités.

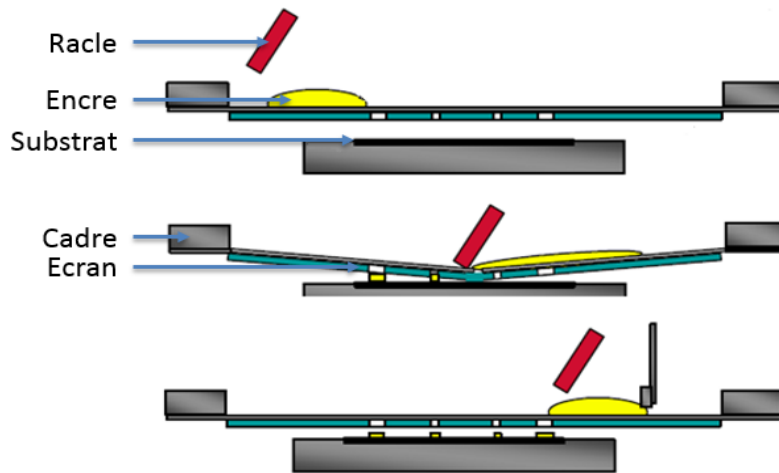


FIGURE 2 : SCHEMA REPRESENTANT LA TECHNIQUE DE SERIGRAPHIE

Après impression, il ne reste plus que le traitement thermique de l'encre pour que les solvants s'évaporent et que l'encre durcisse. Bien évidemment, les propriétés mécaniques et électriques vont dépendre de ce traitement en fonction de la durée et de la température de recuit comme précisé plus tôt. Ces paramètres seront donc suivis afin d'étudier la résistivité de l'encre.

3.2.2 Les pistes conductrices

La figure 3 est une photo du masque d'impression qui sera utilisé pour caractériser les encres. La résistance étant variable en fonction des paramètres morphologiques (longueur, largeur et épaisseur), il est primordial de pouvoir étudier la résistivité selon ces paramètres. C'est pourquoi le masque comporte 5 longueurs de piste (1 cm; 2,5 cm; 5 cm; 10 cm; 15 cm) avec pour chaque longueur 5 largeurs différentes (0,15 mm; 0,25 mm; 0,5 mm; 1 mm; 2 mm). Ce seront les seuls motifs étudiés dans ce stage. Enfin, une caractérisation est correcte seulement si la notion de statistique est présente. C'est pour cela que le

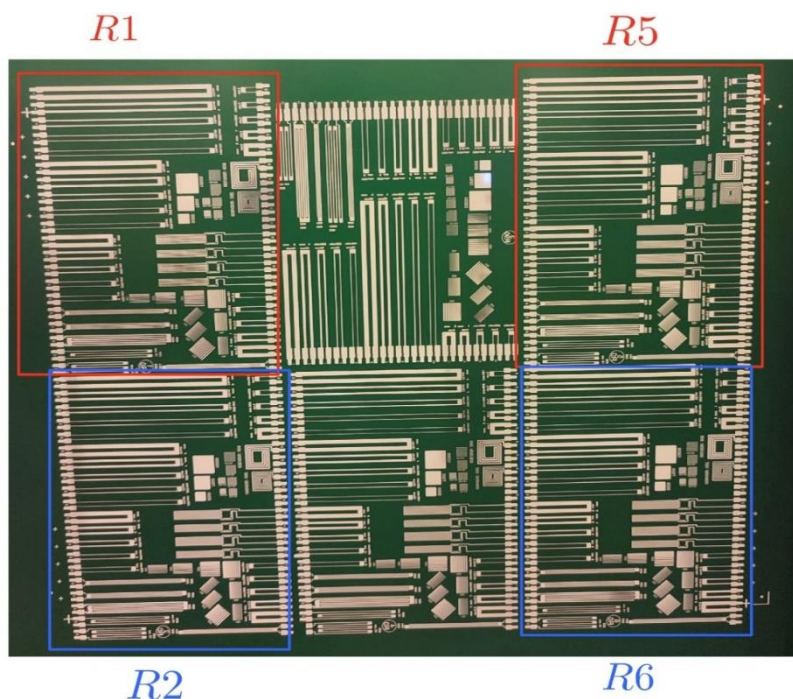


FIGURE 3 : PHOTO DU MASQUE DE SERIGRAPHIE UTILISE

masque est décomposé de deux zones. En effet, le même motif est dupliqué deux fois sur une même zone.

Au préalable, toutes les pistes ont été imprimées par sérigraphie et ont subi un traitement thermique permettant de changer les propriétés électriques. 4 lots sont mis à disposition, leurs paramètres sont reportés dans le tableau suivant (Figure 4):

Batch : B	Température(°C) : T	Numéro de feuille : S	Temps (min) : t
1	120	1 à 5	5
		6 à 10	15
		11 à 15	25
2	120	1 à 5	10
		6 à 10	20
		11 à 15	30
3	100	1 à 5	5
		6 à 10	15
		11 à 15	25
4	100	1 à 5	10
		6 à 10	20
		11 à 15	30

FIGURE 4 : TABLEAU RECAPITULATIF DE REPARTITION DES ECHANTILLONS

Il est à noter que beaucoup de données seront générées dans l'objectif d'avoir une statistique suffisante. C'est pourquoi l'un des enjeux est de bien définir le nom de chaque mesure de manière à les référencer et à les implémenter dans des graphiques d'intérêt. La notation choisie est la suivante : SXBX_IXLX¹. Par exemple, S11B2_L0.15_I5 signifie que c'est une piste ayant une largeur de 0.15 mm et une longueur de 5 cm présente sur la feuille 11 et le lot n°2, donc une température de recuit de 120°C et un temps de recuit de 25 min.

3.2.3 La profilométrie mécanique

Un profilomètre mécanique permet de mesurer l'épaisseur et la largeur d'un motif. Il est composé de deux parties principales. La première partie étant une partie physique mesurant un déplacement latéral et vertical qui correspond respectivement à une largeur ainsi qu'à une épaisseur. Cette mesure est effectuée grâce à une pointe microscopique constituée d'une sphère de diamant d'un rayon de 2µm.

La deuxième partie traduit cette variation physique en un signal électrique afin de le traiter par la suite, c'est le rôle du transducteur.

Les avantages du profilomètre à contact est d'abord le principe mécanique, où la mesure est indépendante aux propriétés optiques de la surface analysée. Le profilomètre mécanique possède une course de mesure pouvant dépasser les 50 cm et donc est adapté à plusieurs types de besoins. Enfin, le profilomètre a l'option d'être programmable afin de démarrer une séquence de mesure répétable sur plusieurs endroits de la feuille.

L'inconvénient principal de ce type d'appareil réside dans le fait que la pointe est directement en contact avec le matériau, ce qui peut donc endommager la surface de ce dernier. Un autre inconvénient est la lenteur de la mesure engendrée par la vitesse du stylet qui est de l'ordre de la centaine de micromètres par seconde.

¹ « S » pour sheet (feuille), « B » pour batch (lot), « l » pour longueur et « L » pour largeur

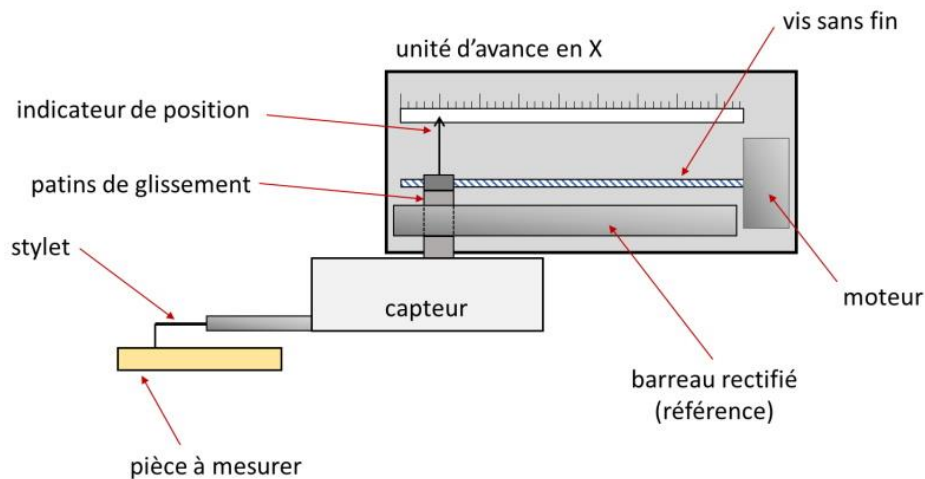


FIGURE 5 : SCHEMA D'UN PROFILOMETRE A CONTACT

Afin d'exploiter au mieux les capacités du profilomètre, l'option programmable sera utilisée. De ce fait, deux mesures par pistes seront effectuées, et seront moyennées grâce au programme développé ultérieurement. Lors de mesures avec le profilomètre, le protocole suivant sera appliqué :

- Déposer la feuille sur une plaque en verre préalablement préparée avec une couche d'adhésif double face et la placer sur le plateau prévu à cet effet avec le cadrant symbolique en haut à gauche.
- Refermer la vitre de protection et allumer la pompe à vide du plateau.
- Lancer le programme intitulé « Caractérisation morphologique VASERELY » et placer les mires de calibrations sur les motifs de croix.
- En fin de programme, exporter les données au format CSV

3.2.4 Banc de mesures électriques

Au vu du nombre d'échantillons préparés, les mesures de résistance seront réalisées avec le banc mis en place au sein de salle de caractérisation électrique, le banc DACTEM. Ce banc a l'avantage de simplifier la mesure, puisqu'une feuille comporte 40 pistes conductrices, et ceci de manière répétable. Afin de mesurer la résistance des pistes, deux options sont possible : un multimètre et un LCR mètre. Le LCR mètre est capable de mesurer l'inductance, la capacité et la résistance. Souvent utilisé pour les composant dont l'impédance varie en fonction de la fréquence, il peut être utilisé comme un ohmmètre. Après réflexion, le choix s'est porté vers le LCR mètre pour plusieurs raisons. Premièrement, le multimètre possède une résistance interne de $20k\Omega$ ce qui pourrait induire des imprécisions car la résistance des pistes à mesurer seront comprises entre $0,1\ \Omega$ et $80\ \Omega$. De plus, le LCR mètre nous permet de mesurer beaucoup plus de paramètres annexe tel que l'inductance, l'impédance et la phase.

Nous créons ensuite notre propre recette de mesure comme définie dans la documentation du DACTEM voir annexe tant. Cette dernière détermine dans quelle gamme de fréquence, avec quelle tension ainsi que les numéros des pistes à mesurer. De plus, comme les pistes étudiées sont linéaires, aucune inductance n'est générée. Les mesures réalisées sont donc l'impédance, la résistance série et parallèle ainsi que la phase en fonction de la fréquence pour 5 points variant entre 4 Hz et 10 kHz.

Après la fin des mesures, le banc de caractérisation enregistre toutes les données dans un dossier contenant toutes les étapes² voulu. Ces dossiers étapes contiennent un fichier CSV par piste.

² Les étapes sont les mesurandes renseigné dans la recette du DACTEM.

De la même manière que pour le profilomètre, afin d'obtenir des résultats s'affranchissant d'un maximum d'erreur, nous mettons en place le protocole suivant :

- Placer un côté de la feuille sur le banc de sorte à ce que toutes les pointes se retrouvent en contact avec les pattes des pistes.
- Inscrire le nom de l'échantillon et sélectionner la bonne recette dans le dossier :
D :\Data\Seq\PIEZO\VASERELY

4 Mesures expérimentales

Cette partie est primordiale et le déroulement du stage en est dépendant. En effet, le but de cette partie est de pouvoir calculer la résistivité des pistes cependant, il faudra commencer par réaliser les mesures morphologiques de toutes les pistes puis les mesures de résistance. Seulement enfin, il sera possible de calculer la résistivité et mettre en évidence les sources d'incertitudes prépondérantes.

4.1 Mesures profilométriques

Cette partie est dédiée à l'étude morphologique des pistes imprimées et l'influence qu'ont certains paramètres sur leur section. C'est une étape importante dans cette étude puisque si la section est mal définie, l'erreur sera directement répercutée sur la résistivité (voir équation (1)). Afin d'effectuer cette caractérisation morphologique, il est primordial d'étudier les paramètres un par un, de cette manière nous pourrons par la suite éliminer les paramètres n'exerçant aucune influence sur la section.

La liste des paramètres variables pour les pistes conductrices est :

- La largeur de piste
- La longueur de piste
- La température et temps de recuit
- L'enchaînement de feuille

C'est à tour de rôle que la variation de section en fonction d'un des paramètres en fixant les autres sera étudiée. Dans un premier temps nous nous sommes intéressés à l'étude de section en fonction de la largeur de piste. L'objectif de cette étude est de montrer que la section est proportionnelle à la largeur de la piste mais aussi de déterminer la précision des mesures. Après avoir tracé les mesures nous pouvons voir sur la figure 6 que la section est bien proportionnelle à la largeur, néanmoins, le R^2 , étant de 98,98%, ceci indique une variation par rapport à la régression linéaire non négligeable. Néanmoins, le métrologue en charge du suivi de la dérive du profilomètre affirme que la résolution de l'ordre de 500 nm, avec une incertitude de $\pm 1,5$ nm. Nous pouvons alors estimer que la variation de la largeur et/ou l'épaisseur des pistes est causée par la résolution de l'impression (Figure 7).

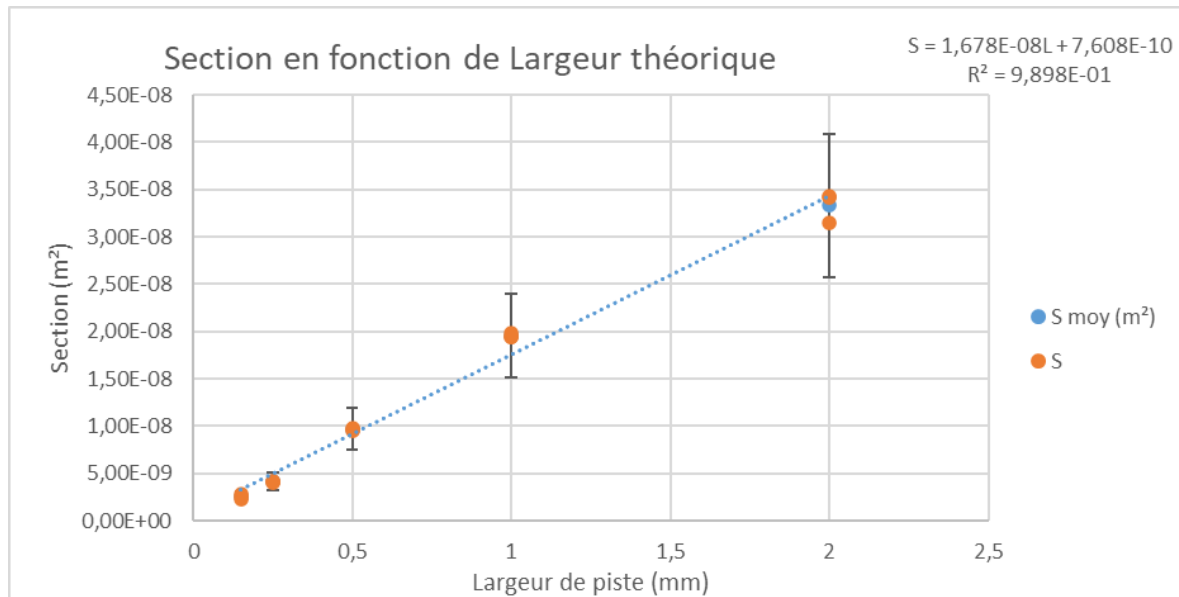


FIGURE 6 : GRAPHIQUE D'ETUDE DE SECTION EN FONCTION DE LA LARGEUR THEORIQUE POUR L'ECHANTILLON S1B2

L'erreur de répétabilité définit la capacité à mesurer le même résultat plusieurs fois, elle est définie par l'équation (2)

$$e(\text{répétabilité}) = \frac{\max(S) - \min(S)}{\text{moyenne}(S)} \quad (2)$$

L'erreur de répétabilité maximum est trouvée pour la largeur de piste de 0,15 mm :

$$e(\text{répétabilité}) = 22,8\%$$

Ce résultat indique que la section, à faible largeur, est très variable. Les résultats ne sont donc pas répétables, ceci est certainement lié à la résolution et à la qualité de l'impression qui génère ainsi des variations de section (Figure 7).

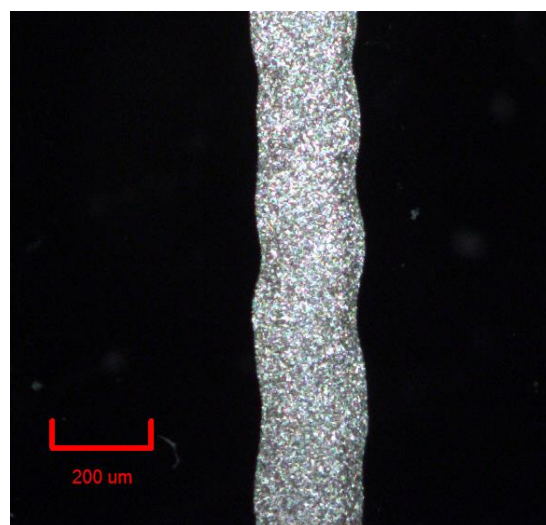


FIGURE 7 : IMAGE REALISEE AU MICROSCOPE OPTIQUE D'UNE PISTE DE 0.15 MM DE LARGEUR

L'erreur de linéarité est également calculée grâce à l'équation (3). Cette erreur représente la différence entre la régression linéaire et les valeurs expérimentales :

$$e(\text{linéarité}) = \frac{\max(S_{\text{prévision}} - S)}{\max(S) - \min(S)} \quad (3)$$

L'erreur associée à la linéarité est de 6,61%, soit, en exécutant la somme quadratique de ces deux erreurs, une erreur de précision totale de : $e_p = 23,76\%$.

Cette erreur servira d'erreur de référence. Si des résultats sont trouvés hors de cette gamme d'erreur, cela voudrait dire qu'une erreur de mesure ou de calcul fait ailleurs n'est pas prise en compte.

En conclusion, l'erreur de répétabilité est en grande partie induite par les pistes conductrices de faible largeur.

Néanmoins, il faut garder à l'esprit que cette erreur est une erreur relative. En effet si nous regardons le graphique de la Figure 8, nous pouvons remarquer que la variation de section la plus grande apparaît pour les pistes de très grande largeur. C'est pour cela que l'erreur de précision est calculé pour chaque largeur. Les résultats de la première étude sont réutilisés pour calculer l'erreur de précision faite pour toute les largeurs de piste, en utilisant la même méthode explicité plus tôt et les équation (2) et (3).

Enfin, certaines valeurs mesurée sont hors gamme d'erreur, ce qui indique que ce paramètre doit être pris en compte.

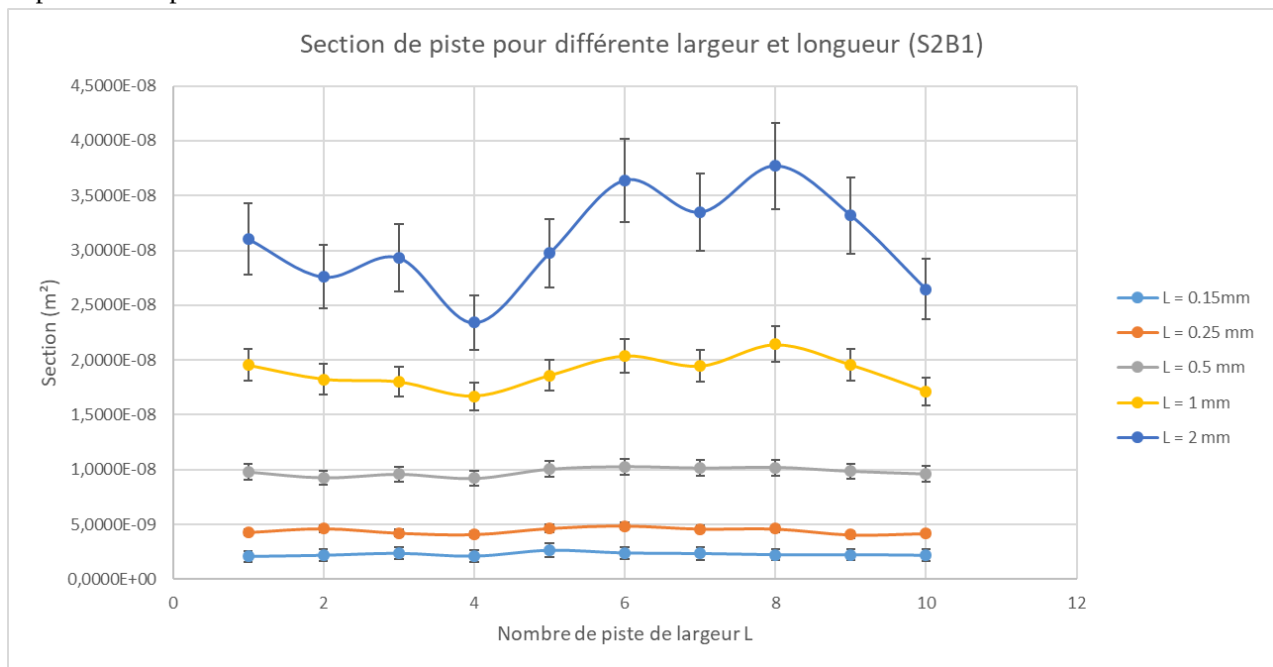


FIGURE 8 : GRAPHIQUE D'ETUDE DE LA SECTION DE PISTE EN FONCTION DE DIFFERENTS ECHANTILLONS

Le paragraphe suivant s'intéresse à la section en fonction de la longueur théorique pour une largeur de 0,5mm. En théorie, la section devrait être constante pour toutes pistes de même largeur à n'importe quelle longueur. Or, le phénomène observable sur le graphique de la figure 9, ci-dessous, est une diminution de la section qui se stabilise à partir de 5cm de longueur.

Cependant, les variations sont comprises dans les barres d'erreur, par conséquent, ce paramètre est classé comme insignifiant sur la mesure.

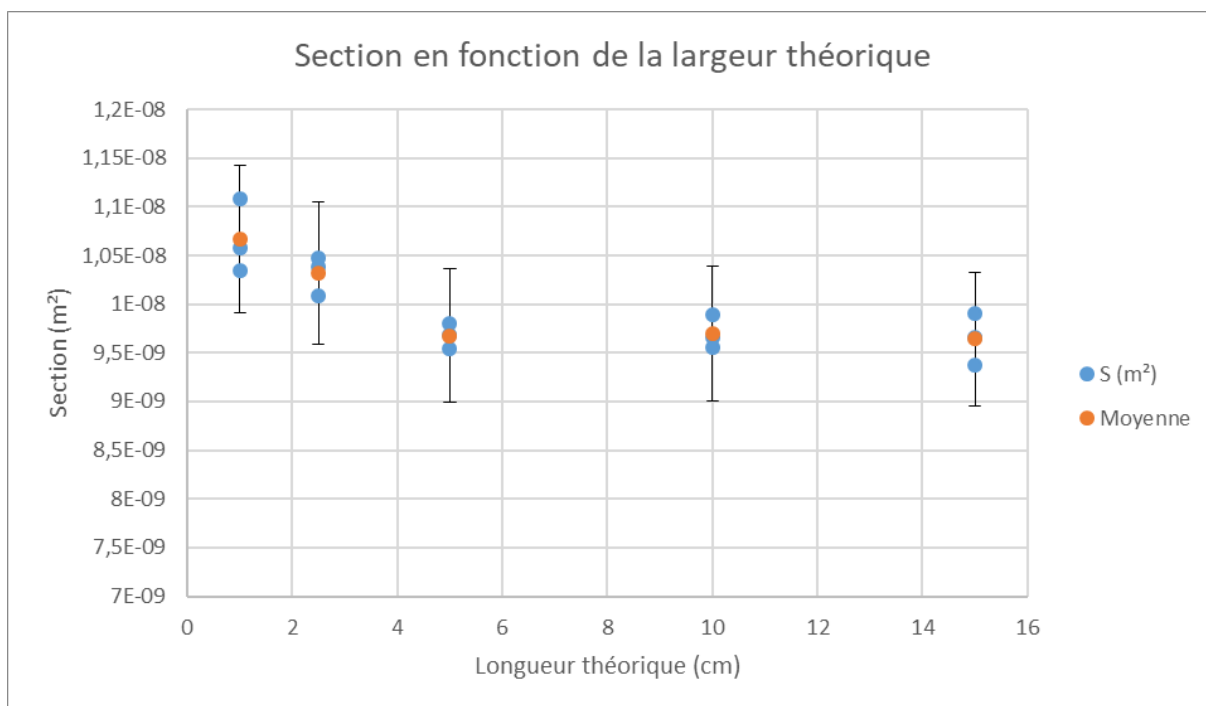


FIGURE 9 : GRAPHIQUE D'ETUDE DE LA SECTION DE PISTE EN FONCTION DE LA LARGEUR THEORIQUE POUR L0.5

Le prochain paramètre étudié est la variation de section en fonction du temps de recuit (Figure 10).

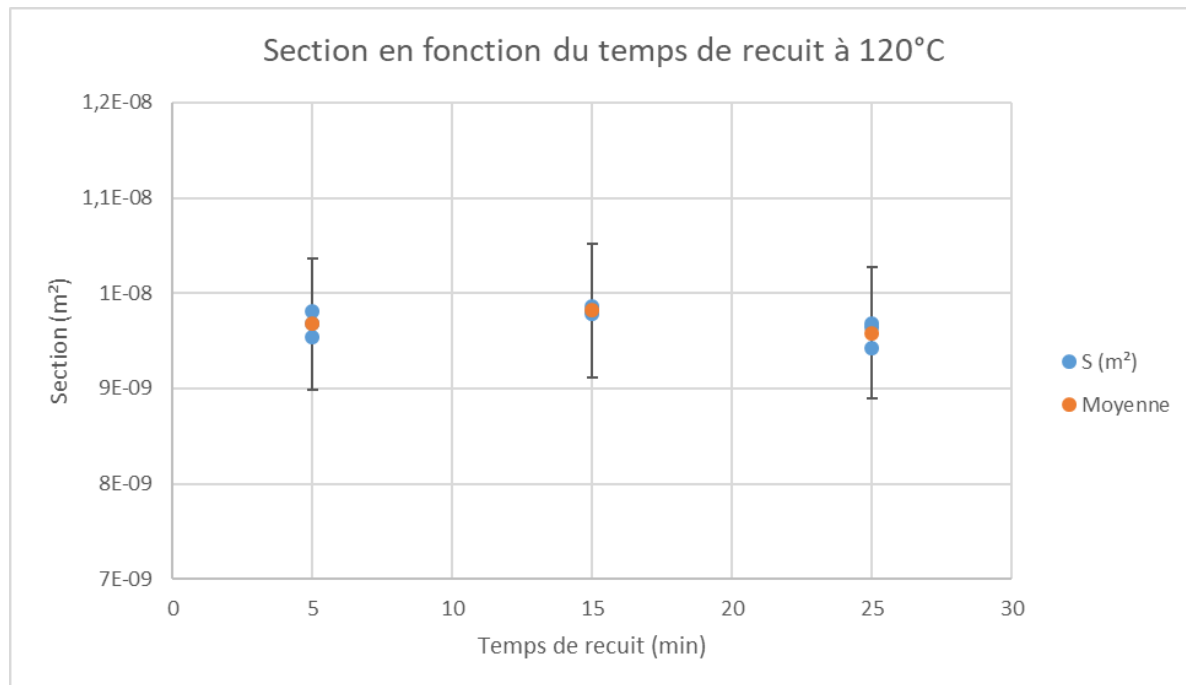


FIGURE 10 : GRAPHIQUE D'ETUDE DE LA SECTION EN FONCTION DU TEMPS DE RECUIT POUR UNE PISTE L0.515

Comme il est possible de voir, les variations de section en fonction du temps de recuit sont minimales. Ces dernières sont de 4,4%. De plus, ces variations se trouvent dans les barres d'erreur, il est ainsi possible d'en conclure que la température et le temps de recuit n'ont aucune influence sur la section.

Enfin, le dernier paramètre étudié est le numéro de feuille. Lors de l'impression par sérigraphie, l'encre déposée sur le masque est en partie consommée, il faut alors rajouter de l'encre à une certaine période pour ne pas voir l'apparition de défauts (manque d'encre, variation de section...). Ce paramètre peut avoir une influence considérable sur la section si l'encre n'est pas remise à une bonne période.

L'objectif ici, est de vérifier que l'épaisseur des pistes est constante sur toutes les feuilles réalisées à une largeur de 0,5mm et pour une longueur de 5cm.

Sur la Figure 11 une diminution de la section après l'impression de 5 feuilles successives est observée. Cette période de 5 feuilles correspond à la période à laquelle nous avons rajouté de l'encre sur le masque. Il pourrait être intéressant de connaître la « consommation » du masque en encre afin de systématiquement rajouter le volume d'encre consommée pour rester à volume constant et supprimer cette erreur.

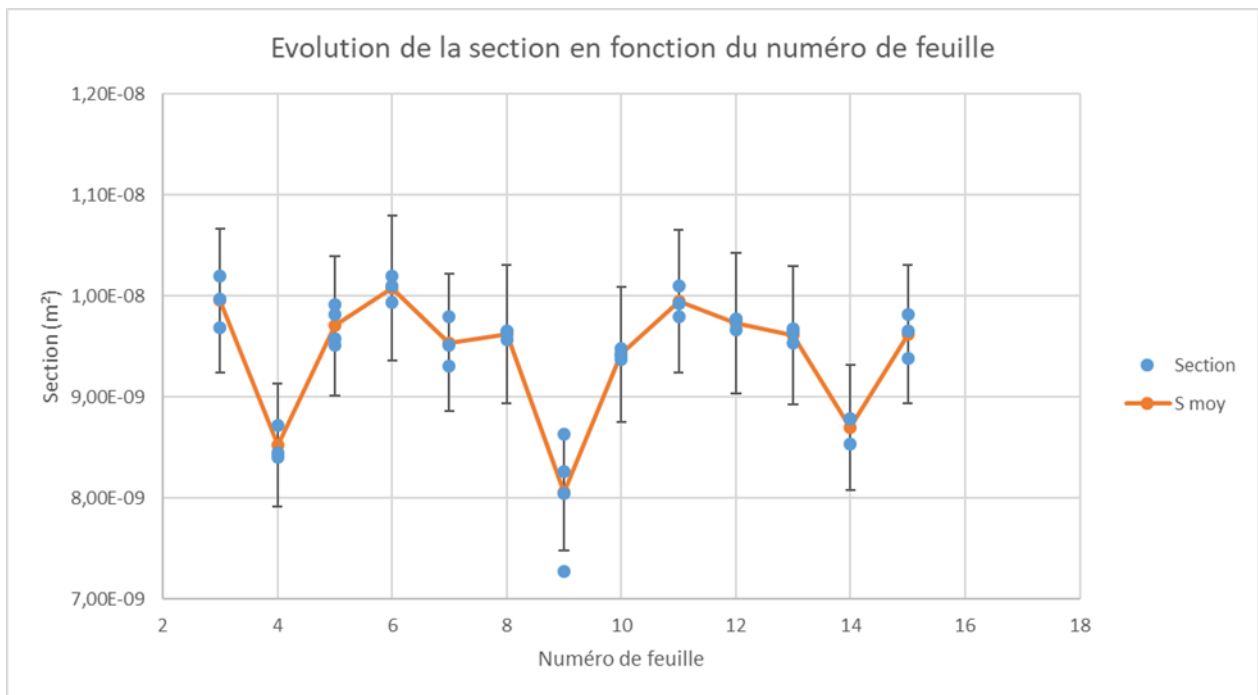


FIGURE 11 : GRAPHIQUE D'ETUDE DE LA SECTION DU NUMERO DE FEUILLE POUR S2B1L0.515

Enfin, les barres d'erreurs ne couvrent pas la totalité des mesures de section. Par conséquent, ces paramètres sont importants et doivent être corrigés pour avoir le moins d'erreur de résistivité induite.

En résumé, la variation de la section des pistes est principalement présente sur les pistes de faible largeur et de faible longueur ainsi que selon le numéro de feuille. Afin de bien caractériser l'encre, ce seront les paramètres majeurs à prendre en compte. La mesure systématique au profilomètre de toutes les pistes permet de limiter les erreurs de variation de section.

4.2 Mesures DACTEM

Après avoir réalisé les mesures sur le banc de caractérisation électrique, le logiciel de traitement statistique JMP est utilisé afin d'analyser finement toutes les données, et d'utiliser des filtres sur les différents paramètres étudiés.

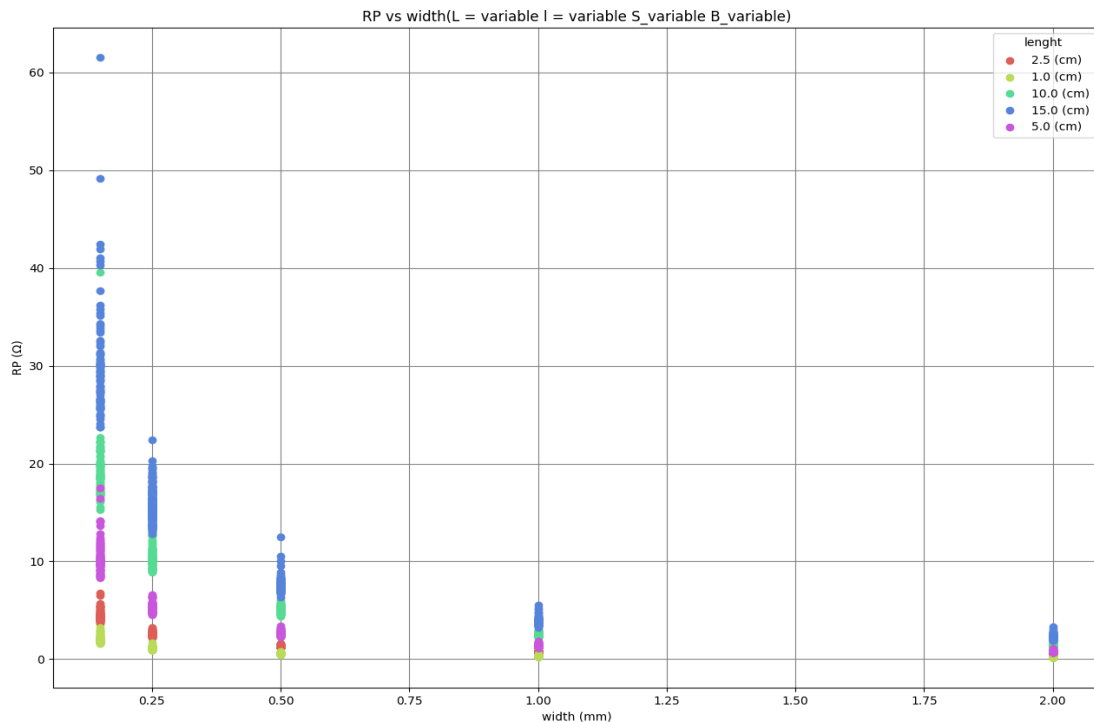


FIGURE 12 : GRAPHIQUE REPRESENTANT LA RESISTANCE EN FONCTION DE LA LARGEUR DE PISTE POUR TOUT PARAMETRES

Après une première analyse, nous observons sur la figure 12 des allures de décroissances exponentielles. De plus, les pistes de faible longueur sont moins affectées par cette décroissance. En théorie, le seul paramètre libre étant la largeur, il aurait été attendu une décroissance de résistance en fonction de la largeur linéaire. En revanche, la section étant déjà corrigée, la seule erreur possible est liée à la mesure de résistance.

La mesure de résistance effectuée par le LCR mètre n'est pas une mesure 4 pointes³. Lors de la mesure, la résistance n'est pas affranchie des résistances des fils qui relient les pointes à l'instrument de mesure. Pour mettre en évidence une telle influence, une résistance étalonnée de 1 ohm avec une erreur de 1% est mesurée avec le banc DACTEM. Afin de connaître la résistance des fils, il restera plus qu'à soustraire la vraie valeur de résistance étalon par la valeur de la résistance trouvée au DACTEM. La longueur des fils étant plus longue pour les pointes plus éloignées, nous réalisons cette mesure 5 fois sur les pointes 1, 10, 20, 30 et 40 afin de calculer une régression linéaire et de corriger la valeur de résistance de mesure. Dans un premier temps la résistance avec une mesure 4 points pour trouver la résistance exacte qui est de 1,01 Ω .

Cette résistance peut paraître insignifiante mais en réalité, certaines pistes que nous mesurons sont du même ordre de grandeur et donc augmenteraient d'un facteur 2 la valeur de résistance/résistivité. Il est donc très important de la prendre en compte.

³ La mesure 4 pointes est effectuée en générant une tension au borne d'un dipôle puis en mesurant l'intensité au borne de ce dernier, le rapport U/I permet de calculer une résistance. De cette manière, la résistance des fils n'est pas comprise dans la mesure.

Afin de corriger la résistance des pistes, nous utiliserons la droite de régression de la figure 13 pour pouvoir calculer automatiquement la résistance des fils en fonction du numéro de la paire de pointes. Cette dernière sera mise en place dans la partie programmation.

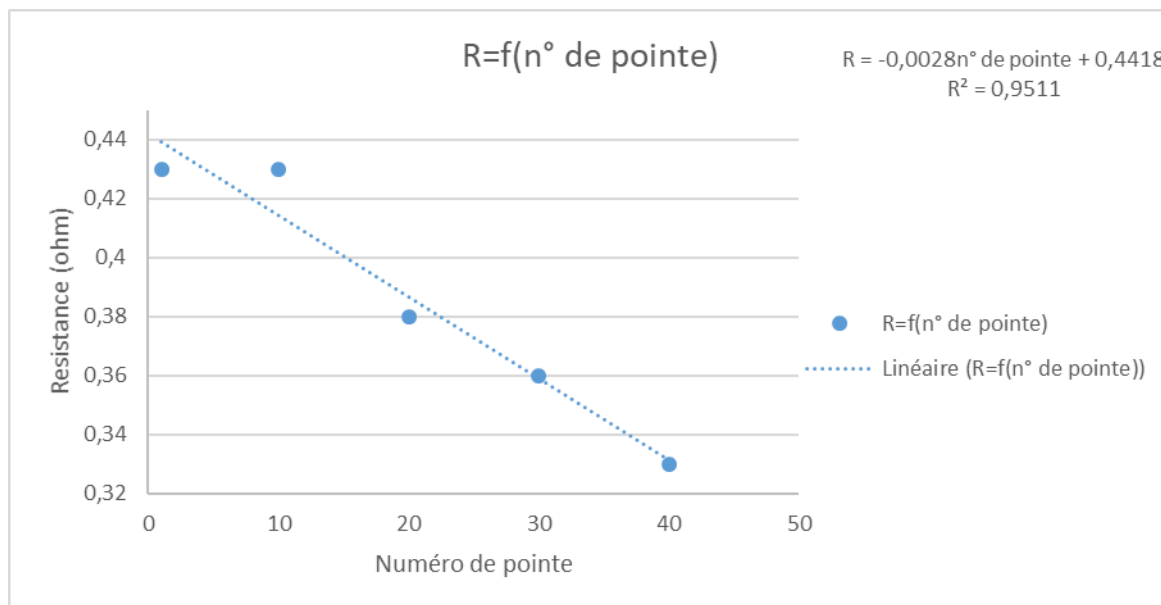


FIGURE 13 : GRAPHIQUE D'ETUDE DE LA RESISTANCE DES FILS EN FONCTION DU NUMERO DE POINTE

De plus, les feuilles étant stockées à l'air libre, l'environnement est par, conséquent, non contrôlé. La résistivité des pistes pourrait ainsi varier en fonction du temps passé à l'air libre avant mesure (Figure 14)

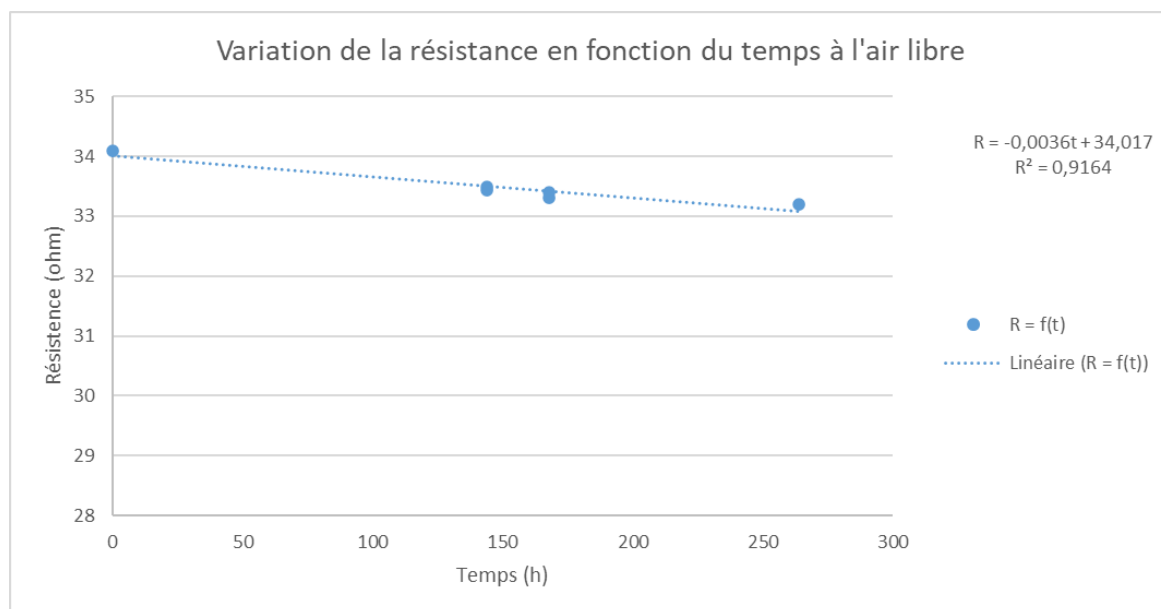


FIGURE 14 : GRAPHIQUE D'ETUDE DE LA RESISTANCE EN FONCTION DU TEMPS EN ENVIRONNEMENT NON CONTROLE

Ce qu'il est observé, est qu'en environ plus de 270h, la résistance d'une piste a diminuée de 1,4 ohm. Cependant, la variation étant de 5%, peut être considéré comme petite par rapport à la résistance initiale,

donc, la variation de résistance à l'air libre est négligeable sur 300h. Il aurait été intéressant de continuer l'étude sur une période plus longue. et avec plus de données.

Maintenant que tous les paramètres définis et que les erreurs liées aux mesures sont corrigées, la prochaine étape est de calculer la résistivité des pistes.

4.3 Estimation de la résistivité

Après avoir récupéré toutes les données morphologiques et électriques des pistes, nous pouvons à présent calculer la résistivité des pistes imprimées telle que formulée dans l'équation (1) grâce aux données exporter préalablement. La résistivité étant une propriété intrinsèque au matériau, celle-ci devrait être constante.

En revanche, ce n'est pas le cas sur le graphique de la Figure 15 En effet, il est possible d'observer une croissance de la résistivité en fonction de l'augmentation de la largeur de piste accentuée pour des pistes de très faibles longueur.

Toutefois, la section étant déjà intégrée pour effectuer un minimum d'erreur, nous pouvons en conclure que l'erreur provient de la mesure de résistance ou du calcul de résistivité.

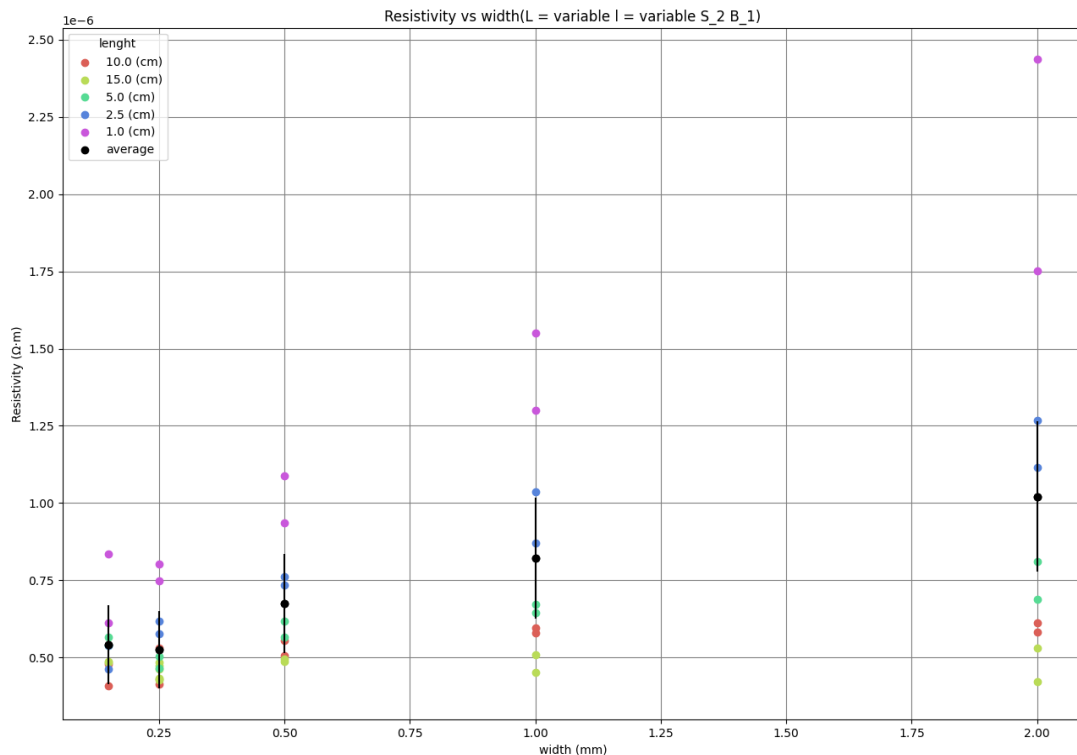


FIGURE 15 : GRAPHIQUE REPRESENTANT LA RESISTIVITE EN FONCTION DE LA LARGEUR POUR UNE FEUILLE : S2B1

Après réflexion, nous remarquons qu'une erreur lors de la conception du masque de sérigraphie a été faite. En effet, lors du dessin du masque, pour effectuer le retour des pistes, un rectangle de dimension différente a été introduit (Figure 16). La piste est alors non uniforme sauf dans le cas de la piste de 2 mm. De plus, nous remarquons que les longueurs de pistes ne sont non plus cohérentes.

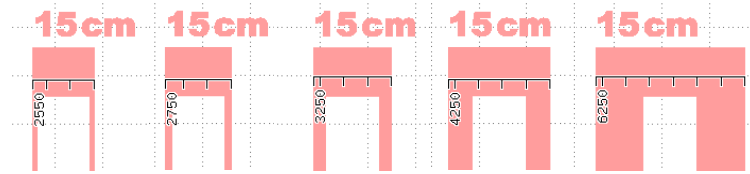


FIGURE 16 : PHOTO DU LAY-OUT DU MASQUE

Ce phénomène pourrait expliquer l'allure que nous pouvons voir sur le graphique de la Figure 15, nous décidons de le corriger en prenant en compte les dimensions réelles détaillées précédemment. Dans le programme du profilomètre nous mettons en place une nouvelle étape afin de mesurer cette section. Le calcul de résistivité devient alors l'équation (4)

$$\rho = \frac{R_T}{\frac{l_{piste}}{S_{piste}} + \frac{l_{rectangle}}{S_{rectangle}}} \quad (4)$$

Nous effectuons alors la même analyse de données que précédemment en changeant le calcul de résistivité et, comme observé sur la figure 17, la croissance observée sur la figure 15 n'est plus visible et que les mesures rentrent dans les barres d'erreurs. Il est possible d'en conclure que l'erreur a bien été corrigée.

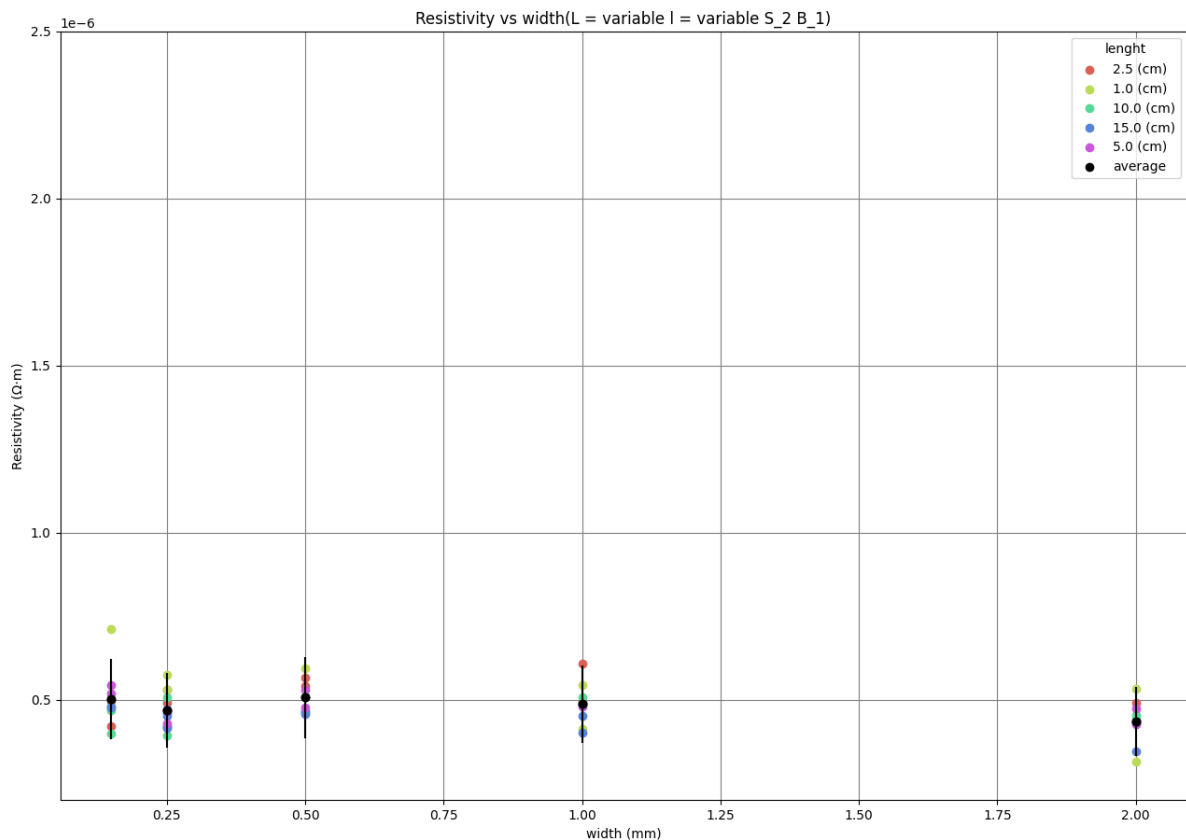


FIGURE 17 : GRAPHIQUE DE LA RESISTIVITE EN FONCTION DE LA LARGEUR POUR LA FEUILLE S1B2

Enfin, pour démontrer que l'énergie ⁴ apportée a bien un impact sur la résistivité de l'encre, nous étudions sur le graphique de la figure 18, la résistivité en fonction de l'énergie apportée. C'est d'ailleurs ce qu'il est montré, la résistivité décroît en fonction de l'énergie apportée. Néanmoins, il était attendu d'observer de variation de résistivité beaucoup plus importante.

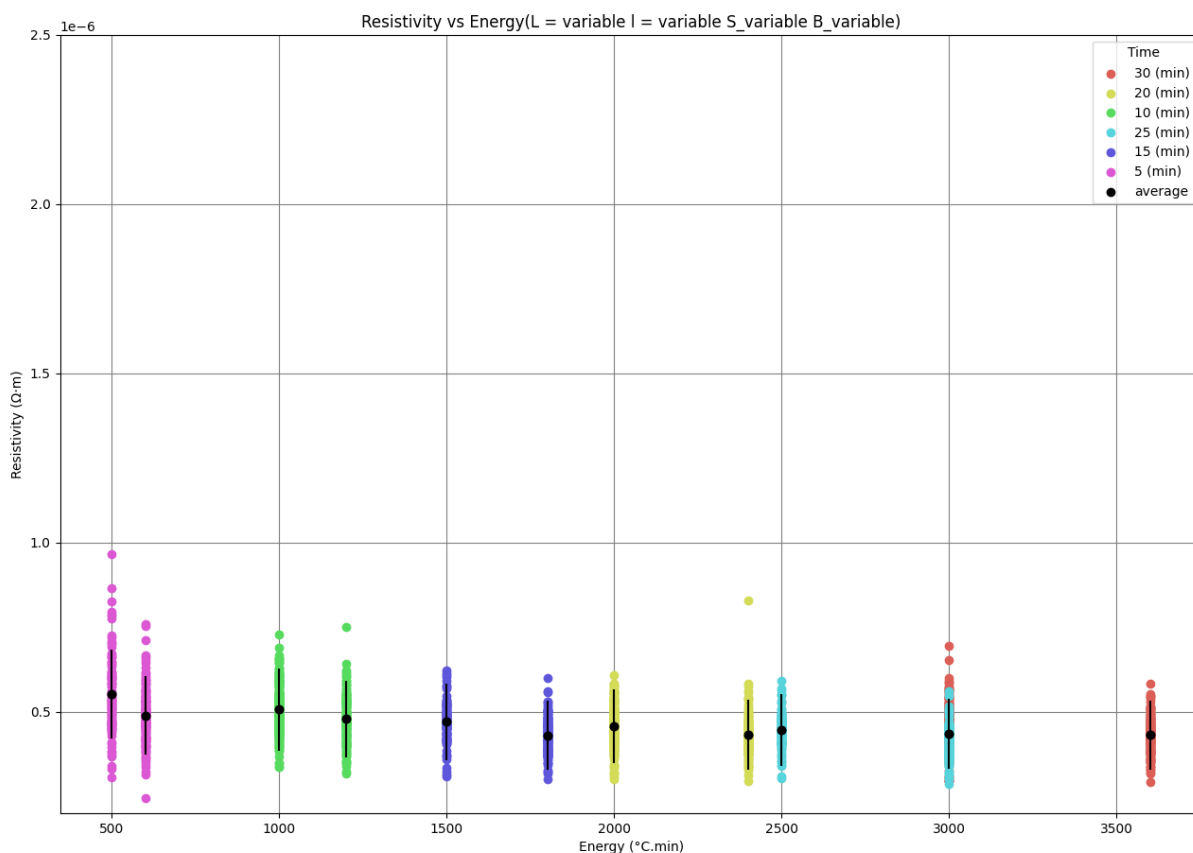


FIGURE 18 : GRAPHIQUE DE LA RESISTIVITE EN FONCTION DU PRODUIT TEMPS/TEMPERATURE POUR TOUTES LES FEUILLES

En revanche, il faut avoir un regard critique sur ce résultat car, par manque de temps, nous n'avons pas pu réaliser l'étude morphologique de multiples feuilles, il manque alors énormément de données pour que les erreurs soit vraiment représentative. L'étude morphologique d'une seule feuille a été alors utilisé pendant toute l'étude. Comme vu dans la partie application profilométrique, la section peut varier d'une feuille à l'autre.

En faisant l'approximation que les sections des pistes d'une feuille sont égales à toutes les autres feuilles, cela induit des erreurs dans le calcul de la résistivité. C'est pourquoi une si grande dispersion de résistivité est observée (Figure 18)

La datasheet du fournisseur de l'encre préconise un recuit à 120°C pendant 10 min pour avoir une résistivité de $4,5 \cdot 10^{-7} \Omega.m$. Avec cette préconisation, nous obtenons une valeur moyenne de $4,787 \cdot 10^{-7} \Omega.m$. Ce qui confirme que nous sommes bien dans les bons ordres de grandeurs. De plus, les résultats sont amenés à évoluer en prenant en compte toutes les sections.

En somme, les paramètres influant sur l'erreur de résistivité sont principalement :

- La section qui, à faible résolution, n'est pas assez répétable.
- La sérigraphie effectuée avec des volumes d'encre variable.
- L'influence des résistances des fils du banc DACTEM

⁴ Ici, l'énergie est en réalité une multiplication du temps de recuit à la température de recuit.

L'influence de l'erreur de conception du masque

De plus, les paramètres qu'il faudra suivre lors de la conception du programme pour réaliser la caractérisation d'une encre sont :

Le produit temps, température de recuit

La largeur des pistes pour différentes longueurs

La section en fonction de la largeur

5 Programmation

Le programme est l'un des points clés pour atteindre l'objectif initial, puisqu'il nous permet de réaliser une étude de caractérisation électrique et morphologique simple. Le programme est basé sur de la logique et il n'existe pas qu'une seule solution à un problème. Il sera alors important d'étudier toutes les solutions disponibles pour pouvoir choisir la solution qui permet d'obtenir un programme court et efficace.

5.1 Les attentes du programme

Lors de l'écriture du programme, il est primordial de ne pas perdre de vue les objectifs, sachant que chaque objectif dépend d'un autre. Ces différents objectifs sont décrits ci-après :

- A. Le premier objectif final est la création d'une base de données afin de comparer la résistivité de différentes encres ou de suivre cette dernière au cours du temps.
- B. Le deuxième objectif final est de réaliser facilement une caractérisation électrique et morphologique.
 - a. Le premier sous-objectif est de pouvoir générer un fichier PDF résumant toutes les informations clés.
 - b. Le suivant est de pouvoir tracer un graphique automatiquement en fonction des paramètres choisis et des filtres appliqués.
 - c. Le dernier sous-objectif est de concaténer des milliers de fichiers morphologiques et électriques pour avoir toutes les informations claires et simples à lire.

Pour développer le programme, il nous suffit alors de suivre les étapes en partant du dernier objectif jusqu'au premier. Les attentes étant écrites, nous pouvons commencer à développer le programme informatique.

5.2 Développement

Dans cette partie nous expliquerons la logique du code. Le code complet annoté en anglais est détaillé en annexe.

Les attentes du programme, listées précédemment, nous permettent de comprendre comment le programme devra être structurée et cela, sans perdre de vue les deux grands objectifs finaux.

De plus, nous avons pris la décision de développer 3 programmes distincts. Un programme sera dédié à la concaténation des fichiers de mesures de résistance, un autre à la concaténation de mesures profilométrique, puis le dernier au calcul de résistivité et de génération de résumé PDF.

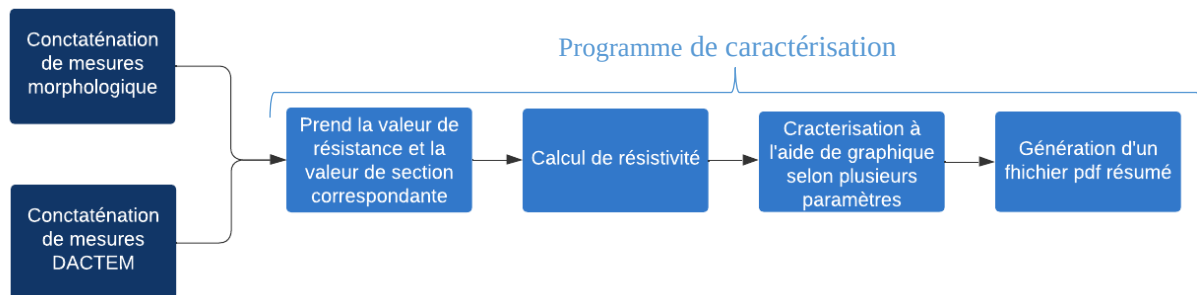


FIGURE 19 : ORGANIGRAMME D'INTERACTION DES PROGRAMMES

Il faut alors dans un premier temps mettre en place les deux programmes de concaténation avant de développer le programme de caractérisation.

Les bases étant posées, commençons alors par la création du programme permettant la concaténation des mesures morphologique et de résistance.

5.2.1 Concaténation de fichiers de données

Pour comprendre pourquoi nous voulons concaténer des fichiers, il faut se référer à la partie dédiée au banc de caractérisation DACTEM. En effet, le DACTEM est capable de créer 40 fichiers Excel comprenant 2 colonnes d'environ 7 lignes, et cela par étape et par piste. Un calcul simple nous permet de comprendre que nous arriverons rapidement à des milliers de fichiers Excel. Cette concaténation est alors un avantage qui nous permet de ranger et classer toutes les données pour ne mélanger aucunes mesures. Le nom des pistes étant déjà défini dans la partie méthodologie, nous réutiliserons cette notation. Plus tôt, nous avons vu que la cinétique a aussi un rôle dans nos mesures, il faudra donc rajouter la date et l'heure à laquelle la mesure de résistance a été effectuée.

5.2.1.1 DACTEM

Quand le programme est lancé, une interface nous permet de choisir les paramètres de la feuille que nous avons mesuré afin de les concaténer. Il est primordial de remplir ces informations avec soin car, plus tard, le programme calculant la résistivité se basera sur les paramètres remplis dans cette étape.

L'utilisateur est alors amené à sélectionner le dossier de mesures DACTEM et à partir de là, l'utilisateur n'a plus rien à faire, le programme s'exécute en allant récupérer les données d'une mesure. Par la suite, le programme vérifie que la résistance n'est pas une résistance infinie ou négative, dans le cas échéant, il la remplace par une erreur. En le répétant autant de fois qu'il y a de dossiers d'étapes et de mesures. Les paramètres étant pré-remplie correctement, toutes les informations liées à la piste se trouve dans le fichier.

A la fin, nous obtenons alors un fichier sous format CSV comportant toutes les informations liées à une piste comme indiqué sur le tableau de la figure 20 ci-dessous.

number	Shape	width	length	Sample	Placement	Temperature	Time	Lot	Bach	Orientation	Sheet	Ink	Energy	RP	RS
1	Straight	2.0	15.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	2.93	2.93
10	Straight	0.15	10.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	26.29	26.29
11	Straight	2.0	5.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	1.31	1.31
12	Straight	1.0	5.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	1.86	1.86
13	Straight	0.5	5.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	3.23	3.23
14	Straight	0.25	5.0	R1	Bas	100	10	6	4	Parallel	1	Bectron	1000.0	5.9	5.9

FIGURE 20 : EXTRAIT DU TABLEAU DE CONCATENATION DE DONNEES DACTEM

L'organigramme ci-dessous permet d'expliquer de manière synthétique et claire le fonctionnement du programme figure (21).

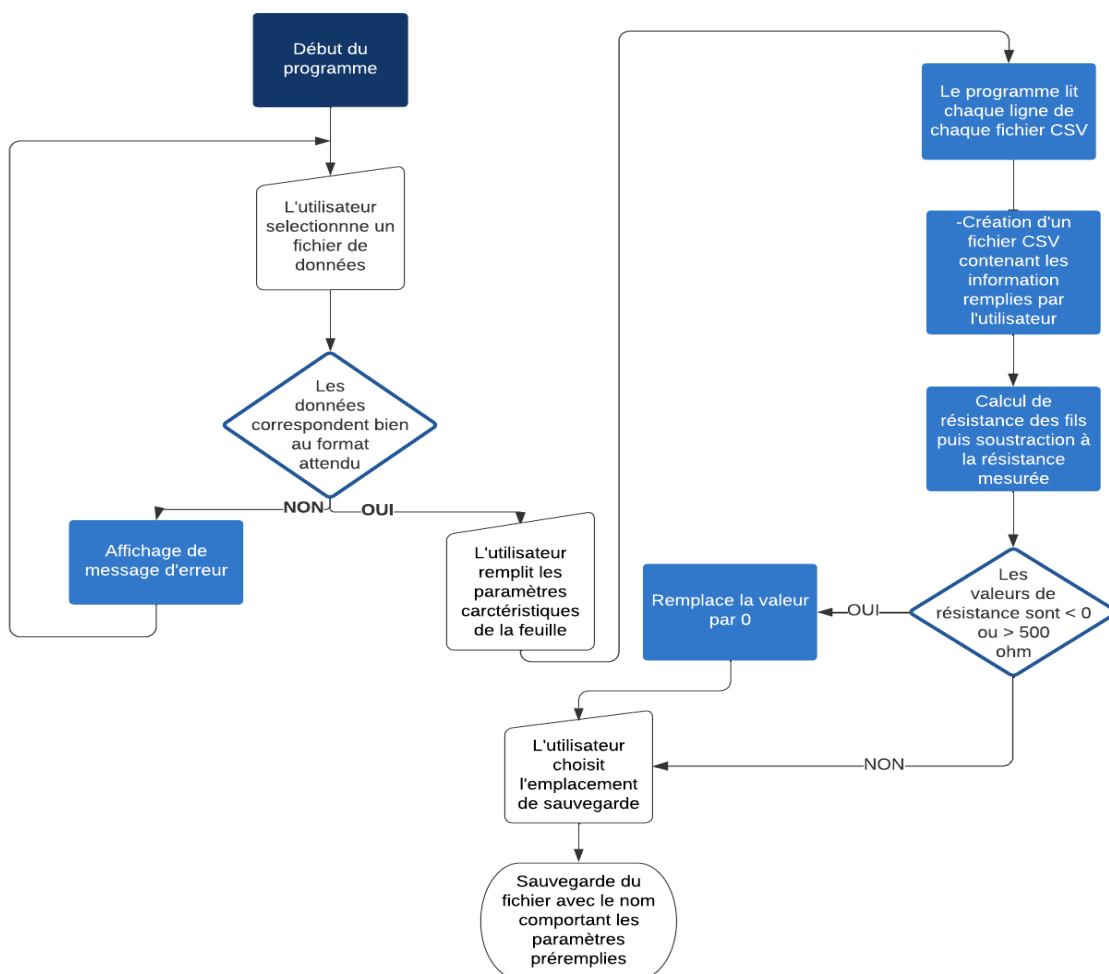


FIGURE 21 : ORGANIGRAMME DE FONCTIONNEMENT DU PROGRAMME DE CONCATENATION DACTEM

5.2.1.2 Profilométrique

Le programme de concaténation des données de profilomètre n'est pas très différent du programme précédent.

En effet, la base est la même : nous demandons à l'utilisateur de rentrer les paramètres de la feuille mesurée. La seule différence est nous devons calculer la section à partir des mesures de largeur et d'épaisseur. Pour ce faire, nous utilisons la méthode d'intégrale décrite dans l'équation (5):

$$\sum \Delta L \times e \quad (5)$$

Avec : ΔL = Pas du profilomètre

e = Epaisseur de l'objet à un point.

Cependant, dans notre cas, le pas du profilomètre (ΔL) correspond à 1 μm . Il suffit alors d'additionner toutes les épaisseurs mesurées pour calculer la section. Les mesures ayant suivie le protocole définie dans la partie méthodologie, après mesure au profilomètre, chaque mesure de section de piste est attribuée à un paramètre de piste. La figure 22 montre sous quelle forme sont stocké les résultats de section.

Parameters	Section
L0.15 10R1	2.0915213946526697e-09
L0.15 10R2	2.189529723383e-09
L0.15 15R1	2.358830511963e-09
L0.15 15R2	2.1147761906372502e-09
L0.15 1R5	2.6222981243655998e-09
L0.15 1R6	2.3855874048397e-09
L0.15 2.5R5	2.3475038075519997e-09
L0.15 2.5R6	2.2373818540727397e-09

FIGURE 22 : EXTRAIT DU TABLEAU CREE PAR LE PROGRAMME DE CONCATENATION PROFILOMETRIQUE

Le programme est répété autant de fois que le nombre de fichiers de mesures fournies. A la fin, un unique fichier CSV comportant toutes les mesures de section des pistes est généré. L'organigramme suivant (figure 23) reprend le fonctionnement du programme :

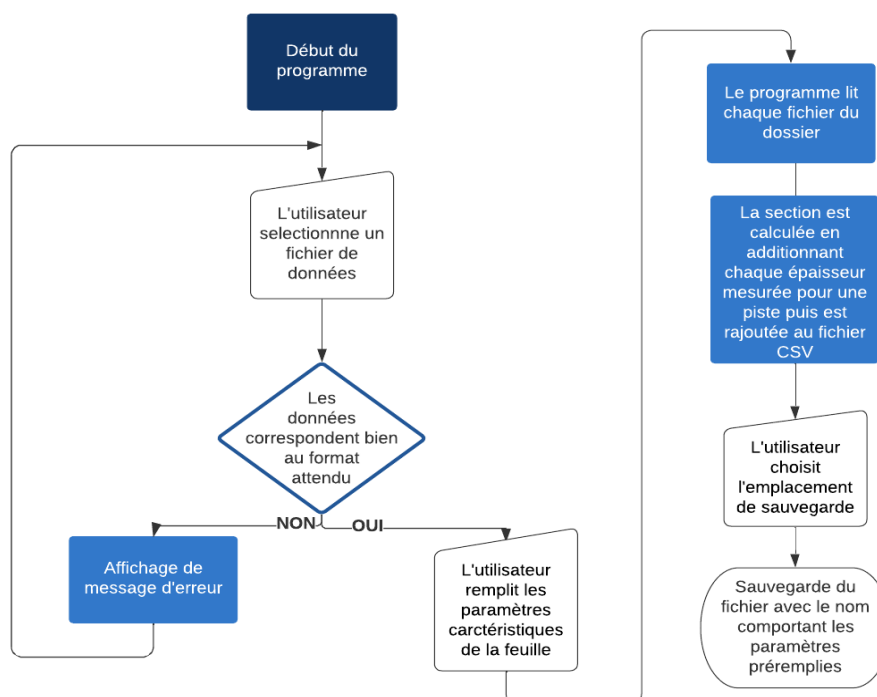


FIGURE 23 : ORGANIGRAMME DE FONCTIONNEMENT DU PROGRAMME DE CONCATENATION PROFILOMETRE

5.2.2 Programme de caractérisation

Le programme de caractérisation a été réalisé en deux parties, la première consiste à réaliser un programme pouvant tracer un graphique en fonction de tous les paramètres (résistivité, section, largeur,

longueur, temps de recuit...). Ce programme nous permet d'identifier les paramètres important à étudier. La deuxième partie est le programme utilisé par la personne souhaitant analyser les performances électriques d'une encre. Ce deuxième programme découle du premier et ne fait ressortir que les éléments clé et important.

- Programme de caractérisation développeur

Comme évoqué à l'instant, ce programme permet d'étudier n'importe quels paramètres de piste. Il comporte alors une interface en face avant nous permettant de sélectionner les données que nous voulons étudier et de filtrer les données selon plusieurs paramètres. L'organigramme détaillant le code peut être trouvé sur le Figure 24

- Programme de caractérisation utilisateur

Ce programme est la réalisation finale du stage, l'objectif étant de regarder toutes les données fournies et d'en faire un résumé pour pouvoir caractériser une encre conductrice. Il faut alors choisir en amont les paramètres que nous voulons étudier et de les définir dans le programme pour permettre à l'utilisateur de réaliser une caractérisation de qualité (Figure 25).

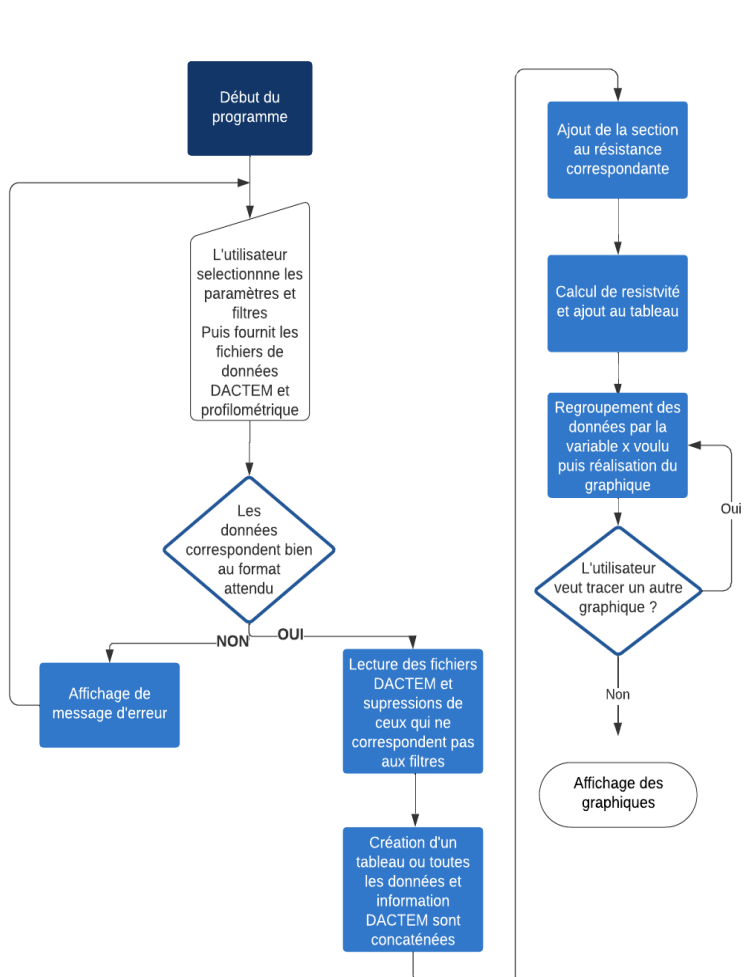


FIGURE 24 : ORGANIGRAMME DE FONCTIONNEMENT DU PROGRAMME DE CARACTERISATION DU DEVELOPPEUR

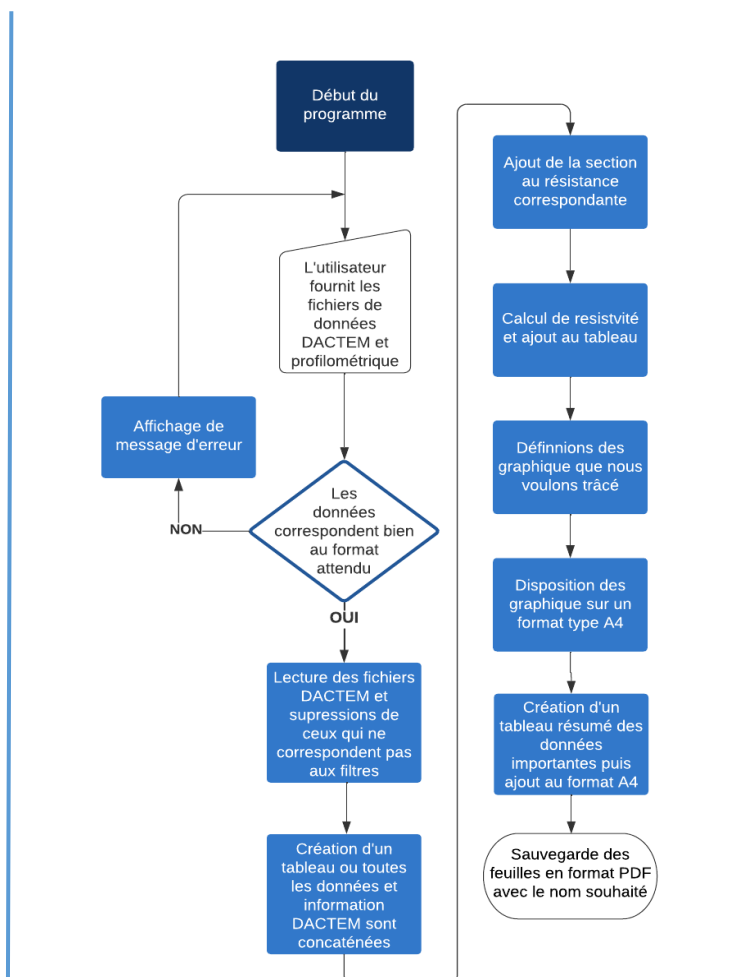


FIGURE 25 : ORGANIGRAMME DE FONCTIONNEMENT DU PROGRAMME DE CARACTERISATION DE L'UTILISATEUR

6 Conclusion

L'objectif de ce stage était la mise en place d'un protocole de caractérisation électrique et morphologique automatique pour des encres conductrices imprimées par sérigraphie. Afin d'y parvenir, la résistivité doit être calculée. Pour la calculer, nous avons mis en place des protocoles de mesures de résistance et de section dans le but d'obtenir des résultats répétables. Après avoir réalisé l'étude de résistivité des pistes, il en est conclu que la résistivité varie très peu selon le temps de recuit et la température de recuit.

Cependant, il est important de nuancer les propos puisque, après analyses des mesures de résistivité, plusieurs paramètres (morphologique et électrique) se sont avérés être variables. En effet, la résistance des fils du banc de mesure n'était pas prise en compte et avait pour effet de doubler la résistivité finale pour les pistes de faible résistance. De plus, le masque n'annonce pas les bonnes valeurs de longueur et de section théorique dû au bout de piste ayant des dimensions différentes. Le masque est donc non adapté pour une bonne caractérisation. Il a donc fallu mettre en place des corrections de valeurs théoriques de longueur. Enfin, la dernière erreur induite est liée au procédé de sérigraphie, en effet la résolution étant un de ses inconvénients, les résultats de section obtenus ne sont pas répétables à faible résolution. La solution retenue est la mise en place de barres d'erreurs sur les graphiques retraçant les données. Après avoir défini toutes les erreurs et paramètres influençant la résistivité, les structurations des programmes ont été réfléchies. Plusieurs programmes ont été développés, le premier est un programme permettant la concaténation de toutes les données de mesure de résistance et le deuxième concatène les données de sections de piste. Après la création de ces deux programmes, un dernier est développé dans le but de récupérer les données concaténées et d'en calculer la résistivité assortie de barres d'erreurs. Même si le programme fonctionne correctement, nous n'avons pas pu réaliser une caractérisation morphologique des pistes complète par manque de temps les erreurs calculées ne sont donc pas totalement représentatives de la réalité.

Lorsque nous évoquons la caractérisation électrique d'encre conductrice, nous pouvons penser que c'est une mesure qui est simple à réaliser et sans difficulté. Cependant, ce stage a permis de démontrer que réaliser une bonne caractérisation électrique précise est complexe. En effet, la section doit être mesurée avec précision car une petite variation peut grandement influencer la valeur de résistivité. Il en est de même pour la mesure de résistance.

En outre, le sujet d'étude est utile pour le service qui est très souvent amené à caractériser des encres. Le service songe néanmoins à réétudier la disposition des pistes sur le masque afin de supprimer certaines erreurs, notamment celles liées aux rectangles faussant la mesure résistive de la piste. Il est aussi important de souligner que ce travail constituera de base pour le prochain masque quand ce dernier sera mis en place.

Le stage de deuxième année de BUT Mesures Physiques est, à mon sens, primordial pour mettre en œuvre les compétences acquises durant la formation et pour pouvoir acquérir de l'expérience. De plus, le stage fut l'occasion pour moi de renforcer l'idée de continuer ma poursuite d'étude en école d'ingénieur en ouvrant mes options aux formations en apprentissage. Enfin, le stage m'a permis de développer de nombreuses compétences tel que le sens de l'adaptation, le développement d'un programme informatique, la mise en place d'un protocole de caractérisation et bien d'autres. Si je continue au sein de la formation BUT Mesures Physiques en 3^{ème} année, je songerai très certainement à effectuer mon stage une nouvelle fois au sein du CEA.

7 **Bibliographie**

- **Sites internet**

Profilomètre :

[Introduction aux profilomètres à contact \(Digital Surf\)](#), consulté le 01/03/2023

[Profilomètre \(Wikipédia\)](#), consulté le 01/03/2023

Technique de sérigraphie :

[Les encres conductrices et leurs applications \(Martin Technologies\)](#), consulté le 28/02/2023

Les encres conductrices imprimées :

[Encre conductrice \(Wikipédia\)](#), consulté le 10/02/2023

- **Revue scientifique**

A. Roberson et al., « Microstructural and Process Characterization of Conductive Traces Printed from Ag Particulate Inks », Journal of Materials, 2011, p963-979, doi:10.3390/ma4060963

S. Merilampi et al., « The characterization of electrically conductive silver ink patterns on flexible substrates », journal of Microelectronics Reliability, 2009, p782–790

K. Jong-Woong et al., « Characterization of direct patterned Ag circuits for RF application », journal of Microelectronic Engineering, 2010, p379–382

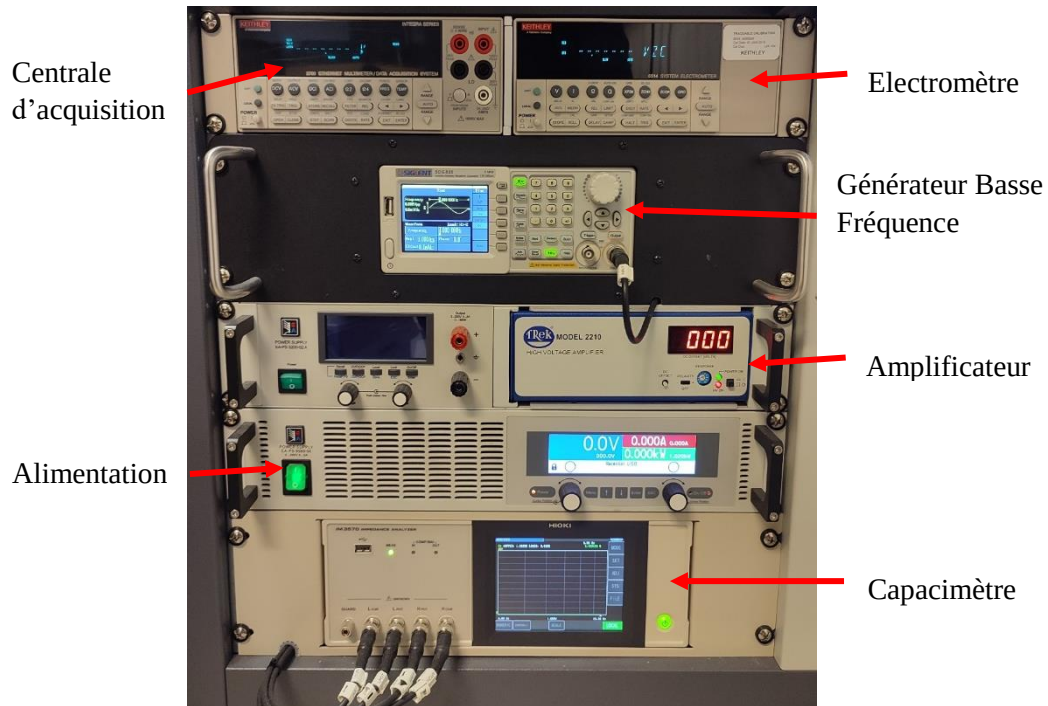
8 Annexes

Table des matières annexes

- Annexe A : Le banc piézoélectrique DACTEM p32
- Annexe B : Le profilomètre p34
- Annexe C : Le programme informatique
 - Programme de caractérisation électrique développeur p34
 - Programme de concaténation DACTEM p47 :
 - Programme de concaténation morphologique p53
- Annexe D : Planning de stage p57

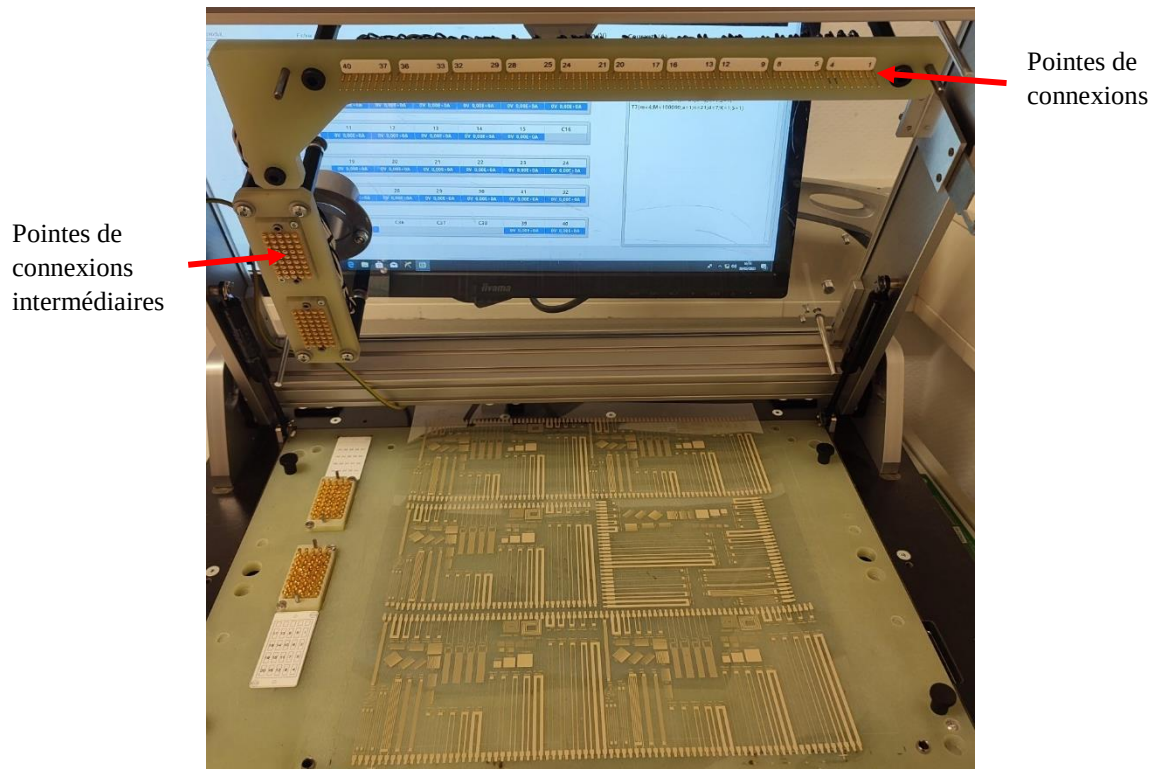
ANNEXE A : Le banc piézoélectrique DACTEM

Le banc d'activation piézoélectrique DACTEM est composé de plusieurs parties, la première partie, la partie basse est principalement composé d'appareils permettant la mesure que nous voulons effectuer.



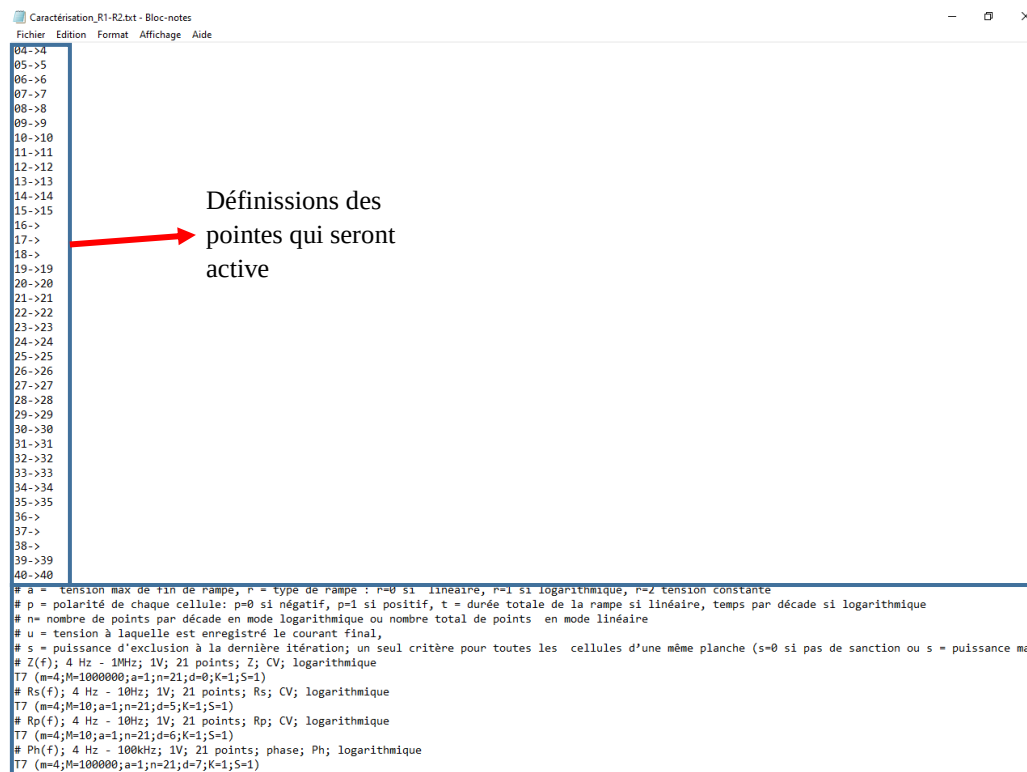
**FIGURE 26 : PHOTOGRAPHIE DE LA PARTIE BASSE DU
BANC DACTEM**

Quant à la partie haute, elle permet la connexion physique entre l'échantillon à mesuré et les pointes de connexion.



**FIGURE 27 : PHOTOGRAPHIE DE LA PARTIE HAUTE DU
BANC DACTEM**

La recette développée est en réalité un fichier texte contenant le numéro des pistes que nous voulons analyser ainsi que les différentes mesures.



```

04->04
05->05
06->06
07->07
08->08
09->09
10->10
11->11
12->12
13->13
14->14
15->15
16->
17->
18->
19->19
20->20
21->21
22->22
23->23
24->24
25->25
26->26
27->27
28->28
29->29
30->30
31->31
32->32
33->33
34->34
35->35
36->
37->
38->
39->39
40->40

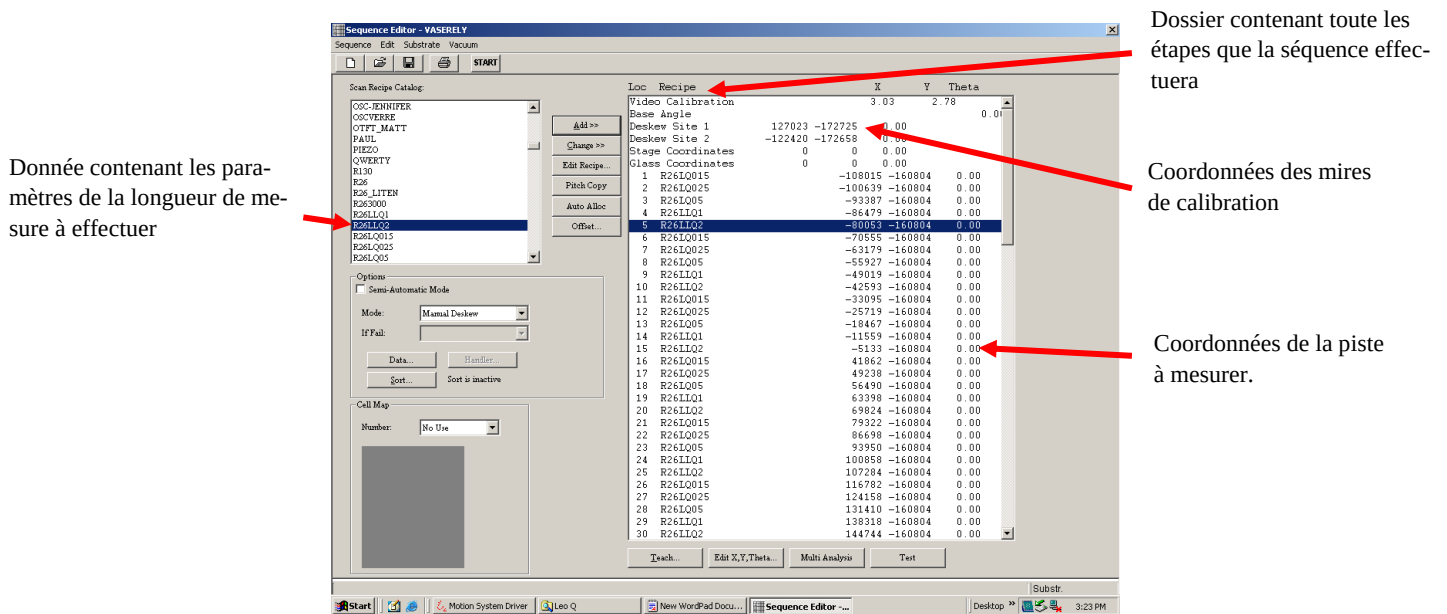
# a = tension max de fin de rampe, r = type de rampe : r=0 si linéaire, r=1 si logarithmique, r=2 tension constante
# p = polarité de chaque cellule: p=0 si négatif, p=1 si positif, t = durée totale de la rampe si linéaire, temps par décade si logarithmique
# n = nombre de points par décade en mode logarithmique ou nombre total de points en mode linéaire
# u = tension à laquelle est enregistré le courant final,
# s = puissance d'exclusion à la dernière itération; un seul critère pour toutes les cellules d'une même planche (s=0 si pas de sanction ou s = puissance ma
# Z(f); 4 Hz - 1MHz; 1V; 21 points; Z; CV; logarithmique
T7 (m=4;M=1000000;a=1;n=21;d=0;K=1;S=1)
# Rs(f); 4 Hz - 10Hz; 1V; 21 points; Rs; CV; logarithmique
T7 (m=4;M=10;a=1;n=21;d=5;K=1;S=1)
# Rp(f); 4 Hz - 10Hz; 1V; 21 points; Rp; CV; logarithmique
T7 (m=4;M=10;a=1;n=21;d=6;K=1;S=1)
# Ph(f); 4 Hz - 100kHz; 1V; 21 points; phase; Ph; logarithmique
T7 (m=4;M=100000;a=1;n=21;d=7;K=1;S=1)

```

FIGURE 28 : FICHIER DE RECETTE DU BANC DE MESURE DACTEM

ANNEXE B : Le profilomètre

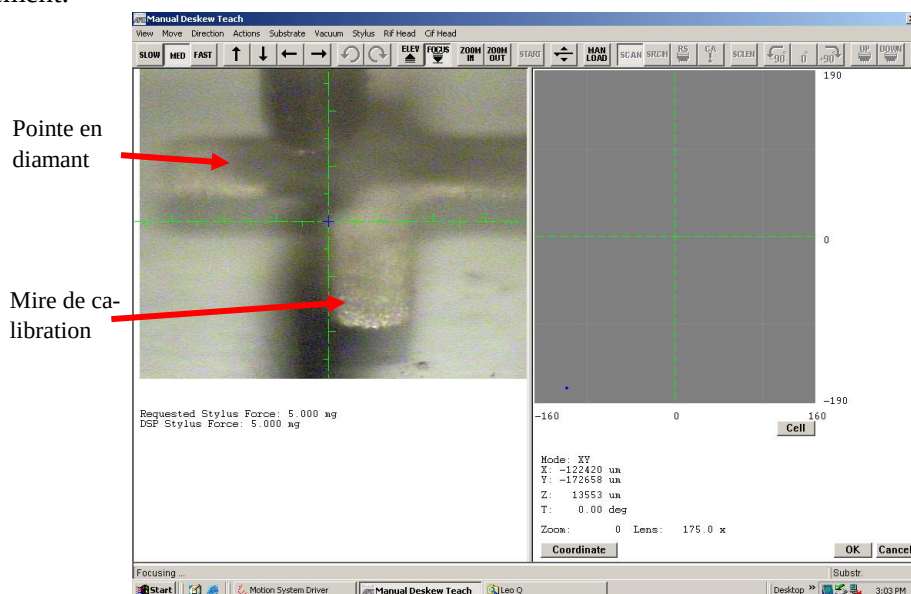
L'option du programme du profilomètre se présente de la manière suivante.



**FIGURE 29 : INTERFACE DU PROGRAMME DE
PROFILOMETRE AUTOMATIQUE**

Une fois les paramètres de longueur de mesure définie, il suffit de rentrer les coordonnées des mires de calibration ainsi que les coordonnées des pistes qui doivent être mesuré. Dans le cas du masque VASERELY, nous réaliserons deux fois une mesure de section sur chaque piste et deux mesures de bout de piste assurant le retour afin de corriger la section. Au total, 102 mesures seront faites, soit l'équivalent de 2 heures de mesures automatique.

Quand l'utilisateur lance ce programme, la pointe en diamant vient se poser en face de la mire de calibration. L'utilisateur n'a donc qu'à corriger la position de la pointe afin qu'elle se retrouve dans le coin supérieur droit de la croix. Une fois la calibration terminée, les mesures s'exécutent automatiquement.



**FIGURE 30 : IMAGE DE LA MESURE DE
PROFILOMETRE**

ANNEXE C :
Les programmes informatiques

Programme de caractérisation électrique développeur :

```
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt
#plt.style.use('ggplot')

import pandas as pd
import os
import seaborn as sns
from pprint import pprint

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from statistics import mean
import cmath, math
import numpy as np
import random
import csv

import tkinter as tk
from tkinter import *
from tkinter import ttk
from tkinter import filedialog
import tkinter.scrolledtext as st

import sys
import re

import scipy.optimize
from scipy.optimize import curve_fit
from scipy.fft import fft, fftfreq

from matplotlib.ticker import AutoMinorLocator

from pathlib import Path

#####

pd.options.mode.chained_assignment = None

def monoExp(x, m, t, b):
    return m * np.exp(-t * x) + b

#Extract the version of the code from the file name
codename=os.path.basename(__file__)

print("Launching",codename)
version=codename[9:].split('.')[0].strip().replace(',','.')

#dictionary

dicunit = {'Section' : 'm²', 'Resistivity' : 'Ω·m', 'width' : 'mm', 'lenght' : 'cm', 'Temperature' : '°C', 'Energy' : '°C.min', 'RP' : 'Ω',
'RS' : 'Ω', 'Time' : 'min'}
```

```

dicolor = { '1' : 'blue', '6' : 'blue', '11' : 'blue', '2' : 'royalblue', '7' : 'royalblue', '12' : 'royalblue', '3' : 'cornflowerblue', '8' :
'cornflowerblue', '13' : 'cornflowerblue', '4' : 'deepskyblue', '9' : 'deepskyblue', '14' : 'deepskyblue', '5' : 'turquoise', '10' :
'turquoise', '15' : 'turquoise'}
disample = { 'R1' : 'red', 'R2' : 'blue', 'R3' : 'green', 'R4' : 'yellow', 'R5' : 'red', 'R6' : 'blue'}
digd = { 'R1' : 'Droite', 'R2' : 'Gauche', 'R5' : 'Droite', 'R6' : 'Gauche'}
dicorrection = {'T120Haut5m' : 84.1, 'T120Haut5g' : 97.4, 'T120Haut5d' : 76.1, 'T120Haut10m' : 99, 'T120Haut10g' : 107.6,
'T120Haut10d' : 92.2, 'T120Haut15m' : 100.9, 'T120Haut15g' : 107.1, 'T120Haut15d' : 95.5, 'T120Haut20m' : 104.9,
'T120Haut20g' : 109.5, 'T120Haut20d' : 97, 'T120Haut25m' : 107.3, 'T120Haut25g' : 111, 'T120Haut25d' : 97.9, 'T120Haut30m'
: 97.9, 'T120Haut30g' : 109, 'T120Haut30d' : 109, 'T120Millieu5m' : 112, 'T120Millieu5g' : 98.5, 'T120Millieu5d' : 77.6,
'T120Millieu10m' : 100, 'T120Millieu10g' : 107.7, 'T120Millieu10d' : 93.6, 'T120Millieu15m' : 101.8, 'T120Millieu15g' : 107.2,
'T120Millieu15d' : 96.9, 'T120Millieu20m' : 105.9, 'T120Millieu20g' : 110.1, 'T120Millieu20d' : 101.7, 'T120Millieu25m' :
108.3, 'T120Millieu25g' : 111.8, 'T120Millieu25d' : 104.6, 'T120Millieu30m' : 109.9, 'T120Millieu30g' : 112.9,
'T120Millieu30d' : 106.5, 'T120Bas5m' : 83.4, 'T120Bas5g' : 93.1, 'T120Bas5d' : 79.7, 'T120Bas10m' : 98.8, 'T120Bas10g' :
104.8, 'T120Bas10d' : 95.3, 'T120Bas15m' : 98.1, 'T120Bas15g' : 102.4, 'T120Bas15d' : 95.3, 'T120Bas20m' : 102.7,
'T120Bas20g' : 106.3, 'T120Bas20d' : 99.6, 'T120Bas25m' : 105.5, 'T120Bas25g' : 108.5, 'T120Bas25d' : 102.3, 'T120Bas30m'
: 107.4, 'T120Bas30g' : 110.1, 'T120Bas30d' : 104, 'T100Haut5m' : 66.5, 'T100Haut5g' : 78.4, 'T100Haut5d' : 58.7,
'T100Haut10m' : 80, 'T100Haut10g' : 87.9, 'T100Haut10d' : 72.9, 'T100Haut15m' : 82.2, 'T100Haut15g' : 87.8, 'T100Haut15d'
: 76.6, 'T100Haut20m' : 86, 'T100Haut20g' : 90.4, 'T100Haut20d' : 81.2, 'T100Haut25m' : 88.2, 'T100Haut25g' : 91.9,
'T100Haut25d' : 84, 'T100Haut30m' : 89.7, 'T100Haut30g' : 93, 'T100Haut30d' : 85.8, 'T100Millieu5m' : 68.5, 'T100Millieu5g'
: 79.1, 'T100Millieu5d' : 62.1, 'T100Millieu10m' : 82, 'T100Millieu10g' : 88.6, 'T100Millieu10d' : 76.5, 'T100Millieu15m' :
83.9, 'T100Millieu15g' : 88.5, 'T100Millieu15d' : 79.7, 'T100Millieu20m' : 87.5, 'T100Millieu20g' : 90.9, 'T100Millieu20d' :
83.9, 'T100Millieu25m' : 89.6, 'T100Millieu25g' : 92.3, 'T100Millieu25d' : 86.5, 'T100Millieu30m' : 91, 'T100Millieu30g' :
93.3, 'T100Millieu30d' : 88.2, 'T100Bas5m' : 71.2, 'T100Bas5g' : 77.4, 'T100Bas5d' : 66.2, 'T100Bas10m' : 83.8, 'T100Bas10g'
: 87.6, 'T100Bas10d' : 79.8, 'T100Bas15m' : 85, 'T100Bas15g' : 87.7, 'T100Bas15d' : 81.8, 'T100Bas20m' : 88.3, 'T100Bas20g'
: 90.4, 'T100Bas20d' : 85.5, 'T100Bas25m' : 90.3, 'T100Bas25g' : 92, 'T100Bas25d' : 87.7, 'T100Bas30m' : 91.6, 'T100Bas30g'
: 93.1, 'T100Bas30d' : 89.2}
dicoven= { '1' : 'Haut', '2' : 'Haut', '3' : 'Millieu', '4' : 'Bas', '5' : 'Bas', '6' : 'Haut', '7' : 'Haut', '8' : 'Millieu', '9' : 'Bas', '10' : 'Bas',
'11' : 'Haut', '12' : 'Haut', '13' : 'Millieu', '14' : 'Bas', '15' : 'Bas'}
digrad = { 'R1' : 'd', 'R2' : 'g', 'R5' : 'd', 'R6' : 'g'}
dic_correction_1 = { 0.15 : 0.255, 0.25 : 0.275, 0.5 : 0.325, 1 : 0.425, 2 : 0.625}
dic_correction_ohm = { 0.15 : 0.7, 0.25 : 0, 0.5 : 0, 1 : 0.0, 2 : 0}
#GUI
class GUI(Tk):
    def __init__(self):
        super().__init__()
        #self.geometry("420x240")      #Tk window size
        w = 785 # width for the Tk root
        h = 450 # height for the Tk root
        ws = self.winfo_screenwidth() # width of the screen
        hs = self.winfo_screenheight() # height of the screen
        # calculate x and y coordinates for the Tk root window
        x = (ws/2) - w/2
        y = (hs/2) - h/2
        self.geometry("%dx%d+%d+%d" % (w, h, x, y))      # set the dimensions of the screen and where it is placed

        self.configure(bg='whitesmoke')
        #style = ttk.Style(self)
        #style.theme_use('clam')

        self.title(codename)      #Tk window title
        self.plot_window()

    def plot_window(self):

        self.grid_columnconfigure(0, minsize=245)
        self.grid_columnconfigure(2, minsize=245)
        self.grid_columnconfigure(4, minsize=245)
        self.grid_columnconfigure(1, minsize=10)

```

```

self.grid_columnconfigure(3, minsize=10)
self.grid_columnconfigure(5, minsize=10)

self.oneplot=IntVar(self)
self.correction=IntVar(self)

#define lists
choicelist=['RS', 'RP', 'Resistivity','width','lenght','Time','Temperature', 'Energy', 'Section']
choicewidth=['variable',2,1,0.5,0.25,0.15]
choicelenght=['variable',15,10,5,2.5,1]
choicetime=['variable',5,10,15,20,25,30]
choicetemp=['variable',100,120]
choiceR=['variable','R1','R2','R3','R4','R5','R6']
choicelot=['variable',1,2,3,4,5,6]
choicebach=['variable',1,2,3,4]
choicesheet=['variable',1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
choiceplace=['variable','Haut','Bas','Gauche','Droite']
choiceink=['variable','Bectron']
choiceshape=['variable','Straight', 'Winding']

#Create blank space to separate the new elements
for j in range(20):
    Frame(self, width=20, height=20, bg='whitesmoke').grid(column=1, row=j)
    Frame(self, width=20, height=20, bg='whitesmoke').grid(column=3, row=j)

#Graph 1 category label
self.label_g1=Label(text="GRAPH 1",font=("Helvetica",12), bg='lightsteelblue')
#self.label_self.rcq.configure(background='light grey')
self.label_g1.grid(row=2, column=0)

self.label_para=Label(text="PARAMETERS",font=("Helvetica",12), bg='lightsteelblue')
self.label_para.grid(row=2, column=4)

#Choice of X - graph 1
self.label_g1xchoice=Label(text="X to plot :", bg='whitesmoke', fg='black')
self.label_g1xchoice.grid(row=3, column=0)
self.g1xchoice = ttk.Combobox(self, width=35)
self.g1xchoice['values']=choicelist
self.g1xchoice.current(3)
self.g1xchoice.grid(row=4, column=0)
self.g1xchoice.configure(state="readonly")

#Choice of Y - graph 1
self.label_g1ychoice=Label(text="Y to plot :", bg='whitesmoke', fg='black')
self.label_g1ychoice.grid(row=5, column=0)
self.g1ychoice = ttk.Combobox(self, width=35)
self.g1ychoice['values']=choicelist
self.g1ychoice.current(0)
self.g1ychoice.grid(row=6, column=0)
self.g1ychoice.configure(state="readonly")

self.separator1=ttk.Separator(self, orient='vertical')
self.separator1.grid(row=2, column=1, rowspan=18, sticky='ns')

#Graph 2 category label

```

```

self.label_g2=Label(text="GRAPH 2",font=("Helvetica",12), bg='lightsteelblue')
#self.label_self.rcq.configure(background='light grey')
self.label_g2.grid(row=2, column=2)

#Choice of X - graph2
self.label_g2xchoice=Label(text="X to plot :", bg='whitesmoke', fg='black')
self.label_g2xchoice.grid(row=3, column=2)
self.g2xchoice = ttk.Combobox(self, width=35)
self.g2xchoice['values']=choicelist
self.g2xchoice.current(1)
self.g2xchoice.grid(row=4, column=2)
self.g2xchoice.configure(state="readonly")

#Fake choice of Y - graph 2
self.label_g2ychoice=Label(text="Y to plot :", bg='whitesmoke', fg='black')
self.label_g2ychoice.grid(row=5, column=2)
self.g2ychoice = ttk.Combobox(self, width=35)
self.g2ychoice['values']=choicelist
self.g2ychoice.current(0)
self.g2ychoice.grid(row=6, column=2)
self.g2ychoice.configure(state="readonly")

#Choice of secondary Y - graph2
self.label_g2y2choice=Label(text="Secondary Y to plot :", bg='whitesmoke', fg='black')
self.label_g2y2choice.grid(row=7, column=2)
self.g2y2choice = ttk.Combobox(self, width=35)
self.g2y2choice['values']=['None','RP','RS','width','lenght','Time','Temperature','Energy']
self.g2y2choice.current(0)
self.g2y2choice.grid(row=8, column=2)
self.g2y2choice.configure(state="readonly")

#Fit graph 2
self.label_g2fitchoice=Label(text="Fit graph 2 :", bg='whitesmoke', fg='black')
self.label_g2fitchoice.grid(row=9, column=2)
self.g2fitchoice = ttk.Combobox(self, width=35)
self.g2fitchoice['values']=('None','Linear fit')
self.g2fitchoice.current(0)
self.g2fitchoice.grid(row=10, column=2)
self.g2fitchoice.configure(state="readonly")

self.separator2=ttk.Separator(self, orient='vertical')
self.separator2.grid(row=2, column=3, rowspan=18, sticky='ns')

#Choosen width
self.label_width=Label(text="Width (mm) :", bg='whitesmoke', fg='black')
self.label_width.grid(row=3, column=4, sticky='w')
self.width = ttk.Combobox(self, width=15)
self.width['values']=choicewidth
self.width.current(0)
self.width.grid(row=4, column=4, sticky='w')
self.width.configure(state="readonly")

#Choosen lenght
self.label_lenght=Label(text="Lenght (cm) :", bg='whitesmoke', fg='black')
self.label_lenght.grid(row=5, column=4, sticky='w')
self.lenght = ttk.Combobox(self, width=15)

```

```

self.lenght['values']=choicelenght
self.lenght.current(0)
self.lenght.grid(row=6, column=4, sticky='w')
self.lenght.configure(state="readonly")

#Choosen temp
self.label_temp=Label(text="Temperature (°C) :", bg='whitesmoke', fg='black')
self.label_temp.grid(row=7, column=4, sticky='w')
self.temp = ttk.Combobox(self, width=15)
self.temp['values']=choicetemp
self.temp.current(0)
self.temp.grid(row=8, column=4, sticky='w')
self.temp.configure(state="readonly")

#Choosen time
self.label_time=Label(text="Time (min) :", bg='whitesmoke', fg='black')
self.label_time.grid(row=9, column=4, sticky='w')
self.time = ttk.Combobox(self, width=15)
self.time['values']=choicetime
self.time.current(0)
self.time.grid(row=10, column=4, sticky='w')
self.time.configure(state="readonly")

#Choosen R
self.label_R=Label(text="Sample n° :", bg='whitesmoke', fg='black')
self.label_R.grid(row=11, column=4, sticky='w')
self.R = ttk.Combobox(self, width=15)
self.R['values']=choiceR
self.R.current(0)
self.R.grid(row=12, column=4, sticky='w')
self.R.configure(state="readonly")

#Choosen lot
self.label_lot=Label(text="Lot n° :", bg='whitesmoke', fg='black')
self.label_lot.grid(row=3, column=4, sticky='e')
self.lot = ttk.Combobox(self, width=15)
self.lot['values']=choicelot
self.lot.current(0)
self.lot.grid(row=4, column=4, sticky='e')
self.lot.configure(state="readonly")

#Choosen bach
self.label_bach=Label(text="Batch n° :", bg='whitesmoke', fg='black')
self.label_bach.grid(row=5, column=4, sticky='e')
self.bach = ttk.Combobox(self, width=15)
self.bach['values']=choicebach
self.bach.current(0)
self.bach.grid(row=6, column=4, sticky='e')
self.bach.configure(state="readonly")

#Choosen sheet
self.label_sheet=Label(text="Sheet n° :", bg='whitesmoke', fg='black')
self.label_sheet.grid(row=7, column=4, sticky='e')
self.sheet = ttk.Combobox(self, width=15)
self.sheet['values']=choicesheet
self.sheet.current(0)
self.sheet.grid(row=8, column=4, sticky='e')
self.sheet.configure(state="readonly")

```

```

#Choosen ink
self.label_ink=Label(text="Ink :", bg='whitesmoke', fg='black')
self.label_ink.grid(row=9, column=4, sticky='e')
self.ink = ttk.Combobox(self, width=15)
self.ink['values']=choiceink
self.ink.current(0)
self.ink.grid(row=10, column=4, sticky='e')
self.ink.configure(state="readonly")

#Choosen place
self.label_place=Label(text="Orientation :", bg='whitesmoke', fg='black')
self.label_place.grid(row=11, column=4, sticky='e')
self.place = ttk.Combobox(self, width=15)
self.place['values']=choiceplace
self.place.current(0)
self.place.grid(row=12, column=4, sticky='e')
self.place.configure(state="readonly")

#Choosen shape
self.label_shape=Label(text="Shape :", bg='whitesmoke', fg='black')
self.label_shape.grid(row=13, column=4, sticky='e')
self.shape = ttk.Combobox(self, width=15)
self.shape['values']=choiceshape
self.shape.current(0)
self.shape.grid(row=14, column=4, sticky='e')
self.shape.configure(state="readonly")

#choosen legend
self.label_legend=Label(text="legend :", bg='whitesmoke', fg='black')
self.label_legend.grid(row=15, column=4, sticky='e')
self.legend = ttk.Combobox(self, width=15)
self.legend['values']=['width','length','Time','Temperature']
self.legend.current(0)
self.legend.grid(row=16, column=4, sticky='e')
self.legend.configure(state="readonly")

#Checkbox to plot only the first graph
self.oneplot_checkbox = tk.Checkbutton(self, text="Only first plot", variable=self.oneplot, cursor="hand2",
bg='whitesmoke',selectcolor='white', fg='black')
self.oneplot_checkbox.grid(row=19, column=0, sticky='ew')

#Checkbox to correct the energy
self.correction_checkbox = tk.Checkbutton(self, text="Temperature correction", variable=self.correction,
cursor="hand2", bg='whitesmoke',selectcolor='white', fg='black')
self.correction_checkbox.grid(row=20, column=0, sticky='ew')

#Button to start plotting

self.go_button = Button(text="PLOT", command=self.plot, cursor="hand2", bg='lightsteelblue', fg='black',
activebackground='darkslateblue', activeforeground='black')
self.go_button.grid(row=20, column=2, sticky='ew')

def plot(self):

    counter=0 #To count how many files have been plotted
    merged_df=pd.DataFrame() #Dataframe where the data will be stored

```



```

corrected_df=pd.DataFrame()
df_average=pd.DataFrame()
todel_list=[]

#Get all the values selected by the operator
g1x=self.g1xchoice.get()
g1y=self.g1ychoice.get()

g2x=self.g2xchoice.get()
g2y=self.g2ychoice.get()
g2y2=self.g2y2choice.get()
g2fit=self.g2fitchoice.get()

oneplot=self.oneplot.get()
correction=self.correction.get()

g1xchoice_list=self.g1xchoice['values']
g1ychoice_list=self.g1ychoice['values']

#get all the parameters picked
gwidth=self.width.get()
glenght=self.lenght.get()
gtime=self.time.get()
gtemp=self.temp.get()
gR=self.R.get()
glot=self.lot.get()
gbach=self.bach.get()
gsheet=self.sheet.get()
gink=self.ink.get()
gplace=self.place.get()
gshape=self.shape.get()
glegend=self.legend.get()

try :
    allfiles=filedialog.askopenfilename(initialdir = "H:/Resume dactem", filetypes=(("csv files" , "*.csv"),), title="Select
one or more files to plot", multiple=True) #ask to select DACTEM data
    print ('-----')
    print ('-----')
except UnicodeDecodeError :
    print("Error : please choose a data file")
if not allfiles:  #If the operator didn't choose a file
    print("No files selected")
    return

profilofiles=filedialog.askopenfilename(initialdir = "H:\Profilo\Section file", filetypes=(("csv files" , "*.csv"),),
title="Select profilometer data", multiple=True)#ask for section file
profilofiles_path=os.path.dirname(profilofiles[0])

foldername=os.path.dirname(allfiles[0])#get the path to dactem data
print("foldername= ",foldername)
print ("Start")

#Define figure and subplots
fig1=plt.figure()

if oneplot==0:
    ax = fig1.add_subplot(211)

```

```

ax2 = fig1.add_subplot(212)

if g2y2!="None":
    ax21 = ax2.twinx()
elif oneplot==1:
    ax = fig1.add_subplot()

#plt.grid(color='grey')

#Detect every file that isn't a data file

allfiles=list(allfiles)

if allfiles==[]:
    plt.close()
    sys.exit("No valid file selected")

color_uniquelist=[]
allfiles2=allfiles
#Read each file and put it into a dataframe
resistivitylist=[]
ploted=[]

final_merged = pd.DataFrame()
concat_df = pd.DataFrame()
for file in allfiles:
    if file in ploted :
        return
    else :
        resistivity=[]
        file=file.replace("\\','")
        filename=file.replace(foldername,"")
        filename=filename[1:]
        filename=filename.rsplit('.')[0]
        print(" Plotting "+filename+"...")
        merged_df=pd.read_csv(file, header=0, sep=',', decimal='.')#read resume file

merged_df = merged_df.drop(merged_df[merged_df.width == 0].index)#remove unknown data
merged_df = merged_df.drop(merged_df[merged_df.lenght == 0].index)
merged_df = merged_df.drop(merged_df[merged_df.RS > 500].index)
merged_df = merged_df.drop(merged_df[merged_df.RS < -500].index)#remove wrong data

# name of parameters of the file

filetime=re.sub("[^0-9]", "", filename.rsplit('_')[3]) #keep only numbers
filetemp=filename.rsplit('_')[4]
filesheet=re.sub("[^0-9]", "", filename.rsplit('_')[6])
filebach=filename.rsplit('_')[1]

#filter the data that don't match parameters and remove it from list

if gwidth != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.width != float(gwidth) ].index)

```

```

if glenght != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.lenght != float(glenght) ].index)

if gtemp != 'variable':
    merged_df = merged_df.drop(merged_df[merged_df.Temperature != int(gtemp) ].index)

if gtime != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Time != int(gtime) ].index)

if gR != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Sample != (gR) ].index)

if glot != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Lot != int(glot) ].index)

if gbach != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Bach != int(gbach) ].index)

if gsheets != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Sheet != int(gsheets) ].index)

if gink != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Ink != gink ].index)

if gplace != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Placement != gplace ].index)

if gshape != 'variable' :
    merged_df = merged_df.drop(merged_df[merged_df.Shape != gshape ].index)

if merged_df.empty : #if dataframe is empty, skip to next file
    print ('Empty dataframe...')

else :
    merged_df=merged_df.sort_values(by=g1x)#sort data per x axes

    #check for unique data
    dsample=merged_df.Sample.unique()
    dtemp=merged_df.Sample.unique()
    dtime=merged_df.Sample.unique()
    dwidth=merged_df.Sample.unique()
    dlenght=merged_df.Sample.unique()

    #get size of unique data
    nsample=dsample.size
    ntemp=dtemp.size
    ntime=dtime.size
    nwidth=dwidth.size
    nlenght=dlenght.size

    dtotal=dsample+dtemp+dtime+dwidth+dlenght

    #read section files

```

```

if len (profilofiles) == 1 :
    section_df=pd.read_csv(profilofiles[0], header=0, sep=',', decimal='.')

else :
    section_df=pd.read_csv(profilofiles_path+'/'+S'+filesheet+fileback+'.csv', header=0, sep=',', decimal='.')#read
section file

for index, row in merged_df.iterrows():
    #get values for each row in one csv
    rawidth=row['width']
    rawlenght=row['lenght']
    rawR=row['Sample']
    resistance=row['RP']

    rawidth_str=str(rawidth)
    rawlenght_str=str(rawlenght)

    if rawidth_str.rsplit('.')[1] == '0' :
        rawidth_str=rawidth_str.rsplit('.')[0]

    if rawlenght_str.rsplit('.')[1] == '0' :
        rawlenght_str=rawlenght_str.rsplit('.')[0]

    #print (section_df.loc[section_df.width == rawidth, 'Section'])
    section = section_df.loc[section_df['Parameters'] == 'L'+rawidth_str+'l'+rawlenght_str+rawR,
'Section'].values[0]#get the section attributed to the row
    if section != 0 :
        #resi=((resistance)*section/(rawlenght*10**(-2)))#calculate the resistivity

        resi=(resistance/(((rawlenght*10**(-2))/(section))+((dic_correction_l[rawidth]*10**(-2))/(3.4*10**(-8)))))
    else :
        resi = 0
    merged_df.loc[index,"Section"]=section#add column Section

    resistivity.append(resi)#add resistivity to list

    merged_df.loc[index,"Resistivity"]=resi#add column Resistivity

    #Correction of energy
    if (correction==1 and (g1x =='Energy' or g2x == 'Energy')):

        correctemp = filetemp+dicovent[filesheet]+filetime+digrad[row['Sample']]

        corrected_df.loc[index,'Energy']=dicorrection[correctemp]

    if (correction==1 and (g1x =='Energy' or g2x == 'Energy')):
        merged_df['Energy']=corrected_df['Energy']

    final_merged=pd.concat([merged_df, final_merged])
    final_merged.reset_index(drop = True, inplace=True)

    concat_df=pd.concat([merged_df, concat_df])
    concat_df.reset_index(drop = True, inplace=True)

```

```

counter=counter+1
print(" File plotted")

if oneplot==0:
    #Plot 2nd graph
    ax2.scatter(merged_df[g2x], merged_df[g2y], color='cyan')

    if g2y2!="None":
        ax21.scatter(merged_df[g2x], merged_df[g2y2], color='red')

    resistivitylist.append(resistivity) #add resistivity to 2nd list to save all data

unique_legend=concat_df[glegend].unique()
size_unique_legend=unique_legend.size
colorlist=sns.color_palette("hls", size_unique_legend)
color_i=0

#plot data
if size_unique_legend > 0 :
    for ilegend in unique_legend :
        concat_df_split = concat_df.drop(concat_df[concat_df[glegend] != ilegend].index)#split dataframe
        ax.scatter(concat_df_split[g1x] ,concat_df_split[g1y], color=colorlist[color_i], label = str(ilegend)+'
('+dicunit[glegend]+')')
        color_i=color_i+1

#average local value
unique_x_list=concat_df[g1x].unique()
for xax in unique_x_list :
    local_average_df = concat_df.drop(concat_df[concat_df[g1x] != xax].index)
    average = local_average_df.loc[:, g1y].mean()
    df_average.loc[xax, 'xax']=xax
    df_average.loc[xax, 'average']=average
if (g1y != 'RP' or g1y != 'RS') :
    ax.scatter(df_average['xax'], df_average['average'], color = 'black', label = 'average')
    plt.errorbar(df_average['xax'], df_average['average'], yerr=df_average['average']*0.238, fmt="o", color = 'black')

resistivitylist = [x for x in resistivitylist if x != []] #remove empty lists
resistivitylist = [item for sublist in resistivitylist for item in sublist] #flatten to get one list

if g1y == "Resistivity" :
    print ("resistivity : ",resistivitylist)
    print ('max = ', max(resistivitylist))
    print ('min = ', min(resistivitylist))
    print ('moyenne = ', sum(resistivitylist)/len(resistivitylist))
    print ('Erreur Section : ', ((max(resistivitylist)-min(resistivitylist))/max(resistivitylist)*100), '%')

if len(allfiles)==0:
    plt.close()
    print("Nothing to plot")

else:
    #Display the plot
    #Main plot

```

```

ax.set(xlabel=g1x+' ('+dicunit[g1x]+'), ylabel=g1y+' ('+dicunit[g1y]+'))
leg1=ax.legend(loc='best')
leg1.set_in_layout(False)
ax.set_title(g1y+' vs '+g1x+'(L = '+gwidth+' l = '+glenght+' S_ '+gsheet+' B_ '+gbach+'), color = 'black')
ax.legend(title=glegend)
ax.grid()
ax.set_axisbelow(True)

if oneplot==0:
    #Fit graph 2
    if g2fit=='Linear fit' :
        #Charge vs ?

        x=final_merged[g2x].astype(float)
        y=final_merged[g2y].astype(float)
        fit=np.polyfit(x, y, 1)
        fitplot=np.poly1d(fit)#equation linear fit
        b=fitplot(0)
        a= fitplot(1)-b
        print ('a = ', a)
        print ('b = ', b)
        ax2.plot(final_merged[g2x],fitplot(final_merged[g2x]),linestyle='dashed', label=str(fitplot), color='cyan')

    if g2y2!='None':
        #Force vs measurement number (on same graph)
        y21=merged_df[g2y2].astype(float)
        fit=np.polyfit(x, y21, 1)
        fitplot=np.poly1d(fit)
        ax21.plot(merged_df[g2x],fitplot(merged_df[g2x]),linestyle='dashed', label=str(fitplot), color='red')
        h2, l2 = ax2.get_legend_handles_labels()
        h21, l21 = ax21.get_legend_handles_labels()
        leg221=ax21.legend(h2+h21, l2+l21, loc='best')
        leg221.set_in_layout(False)
    else:
        leg2=ax2.legend(loc='best')
        leg2.set_in_layout(False)

#Second plot
ax2.set(xlabel=g2x+' ('+dicunit[g2x]+'), ylabel=g2y+' ('+dicunit[g2y]+'))
ax2.set_facecolor("black")
ax2.grid()
ax2.set_axisbelow(True)

if g2y2!="None":
    ax21.set(xlabel=g2x+' ('+dicunit[g2x]+'), ylabel=g2y2+' ('+dicunit[g2y2]+'))
    ax21.xaxis.label.set_color('black')
    ax21.yaxis.label.set_color('red')
    ax21.tick_params(axis='x', colors='black')
    ax21.tick_params(axis='y', colors='black')
    ax2.set_title("{} and {} vs {}".format(g2y,g2y2,g2x))
else:
    ax2.set_title("{} vs {}".format(g2y,g2x))

#Fullscreen
#mng = plt.get_current_fig_manager()
mng = fig1.canvas.manager
mng.window.state("zoomed")

```

```

#Figure layout stuff
plt.grid(color='grey')
#fig1.tight_layout()
fig1.subplots_adjust(left=0.052, bottom=0.064, right=0.995, top=0.96, wspace=0.27, hspace=0.22)

print("Everything has been plotted \n")
#plt.ylim(.)
ax.set_ylim([0.0000002, 0.0000025])
plt.show()
print (counter, " files plotted")

if __name__ == '__main__':
    window = GUI()
    window.mainloop()

```

Programme de concaténation DACTEM :

```

import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
plt.style.use('ggplot')

import pandas as pd
import os
import seaborn as sns
from pprint import pprint

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from statistics import mean
import cmath, math
import numpy as np
import random
import csv

import tkinter as tk
from tkinter import *
from tkinter import ttk
from tkinter import filedialog
import tkinter.scrolledtext as st

import sys
import re

import scipy.optimize
from scipy.optimize import curve_fit
from scipy.fft import fft, fftfreq

from matplotlib.ticker import AutoMinorLocator

pd.options.mode.chained_assignment = None

def monoExp(x, m, t, b):
    return m * np.exp(-t * x) + b

dwidth123456 = { '1' : 2, '2' : 1, '3' : 0.5, '4' : 0.25, '5' : 0.15, '6' : 2, '7' : 1, '8' : 0.5, '9' : 0.25, '10' : 0.15, '11' : 2, '12' : 1, '13' : 0.5,
'14' : 0.25, '15' : 0.15, '16' : 0, '17' : 0, '18' : 0, '19' : 0.25, '20' : 0.25, '21' : 2, '22' : 1, '23' : 0.5, '24' : 0.25, '25' : 0.15, '26' : 2, '27'

```

```

: 1, '28': 0.5, '29': 0.25, '30': 0.15, '31': 2, '32': 1, '33': 0.5, '34': 0.25, '35': 0.15, '36': 0, '37': 0, '38': 0, '39': 0.25, '40': 0.25}
dlenght12456 = {'1': 15, '2': 15, '3': 15, '4': 15, '5': 15, '6': 10, '7': 10, '8': 10, '9': 10, '10': 10, '11': 5, '12': 5, '13': 5, '14': 5, '15': 5, '16': 0, '17': 0, '18': 0, '19': 15, '20': 15, '21': 15, '22': 15, '23': 15, '24': 15, '25': 15, '26': 10, '27': 10, '28': 10, '29': 10, '30': 10, '31': 5, '32': 5, '33': 5, '34': 5, '35': 5, '36': 0, '37': 0, '38': 0, '39': 15, '40': 15}

dlenght4h={'2': 2.5, '3': 2.5, '4': 2.5, '5': 2.5, '6': 2.5, '13': 0, '15': 0, '16': 1, '17': 1, '18': 1, '19': 1, '20': 1}
dwidth4h={'2': 2, '3': 1, '4': 0.5, '5': 0.25, '6': 0.15, '13': 0, '15': 0, '16': 2, '17': 1, '18': 0.5, '19': 0.25, '20': 0.15}

dwidth1256h = { '2': 2, '3': 1, '4': 0.5, '5': 0.25, '6': 0.15, '13': 0, '15': 0, '16': 2, '17': 1, '18': 0.5, '19': 0.25, '20': 0.15, '22': 2, '23': 1, '24': 0.5, '25': 0.25, '26': 0.15, '33': 0, '35': 0, '36': 2, '37': 1, '38': 0.5, '39': 0.25, '40': 0.15}
dlenght1256h= { '2': 2.5, '3': 2.5, '4': 2.5, '5': 2.5, '6': 2.5, '13': 0, '15': 0, '16': 1, '17': 1, '18': 1, '19': 1, '20': 1, '22': 2.5, '23': 2.5, '24': 2.5, '25': 2.5, '26': 2.5, '33': 0, '35': 0, '36': 1, '37': 1, '38': 1, '39': 1, '40': 1}

dlenght3g = {'1': 10, '2': 10, '3': 10, '4': 10, '5': 10, '6': 15, '7': 15, '8': 15, '9': 15, '10': 15, '11': 1, '12': 1, '13': 1, '14': 1, '15': 1}
dlenght3d = {'1': 2.5, '2': 2.5, '3': 2.5, '4': 2.5, '5': 2.5, '6': 5, '7': 5, '8': 5, '9': 5, '10': 5, '11': 0, '12': 0, '13': 0, '14': 0, '15': 15}

dwidth3d={'1': 2, '2': 1, '3': 0.5, '4': 0.25, '5': 0.15, '6': 2, '7': 1, '8': 0.5, '9': 0.25, '10': 0.15, '11': 2, '12': 1, '13': 0.5, '14': 0.25, '15': 0.25}

dorient = {"R1R2": "Parallel", "R3": "Perpendicular", "R4": "Parallel", "R5R6": "Parallel"}

dshape12456b = {'1': 'Straight', '2': 'Straight', '3': 'Straight', '4': 'Straight', '5': 'Straight', '6': 'Straight', '7': 'Straight', '8': 'Straight', '9': 'Straight', '10': 'Straight', '11': 'Straight', '12': 'Straight', '13': 'Straight', '14': 'Straight', '15': 'Straight', '16': 'Straight', '17': 'Straight', '18': 'Straight', '19': 'Winding', '20': 'Winding', '21': 'Straight', '22': 'Straight', '23': 'Straight', '24': 'Straight', '25': 'Straight', '26': 'Straight', '27': 'Straight', '28': 'Straight', '29': 'Straight', '30': 'Straight', '31': 'Straight', '32': 'Straight', '33': 'Straight', '34': 'Straight', '35': 'Straight', '36': 'Straight', '37': 'Straight', '38': 'Straight', '39': 'Winding', '40': 'Winding'}
dshape3d = {'1': 'Straight', '2': 'Straight', '3': 'Straight', '4': 'Straight', '5': 'Straight', '6': 'Straight', '7': 'Straight', '8': 'Straight', '9': 'Straight', '10': 'Straight', '11': 'Straight', '12': 'Winding', '13': 'Straight', '14': 'Straight', '15': 'Winding'}
dshapelse = {'1': 'Straight', '2': 'Straight', '3': 'Straight', '4': 'Straight', '5': 'Straight', '6': 'Straight', '7': 'Straight', '8': 'Straight', '9': 'Straight', '10': 'Straight', '11': 'Straight', '12': 'Straight', '13': 'Straight', '14': 'Straight', '15': 'Straight', '16': 'Straight', '17': 'Straight', '18': 'Straight', '19': 'Straight', '20': 'Straight', '21': 'Straight', '22': 'Straight', '23': 'Straight', '24': 'Straight', '25': 'Straight', '26': 'Straight', '27': 'Straight', '28': 'Straight', '29': 'Straight', '30': 'Straight', '31': 'Straight', '32': 'Straight', '33': 'Straight', '34': 'Straight', '35': 'Straight', '36': 'Straight', '37': 'Straight', '38': 'Straight', '39': 'Straight', '40': 'Straight'}

#GUI
class GUI(Tk):
    def __init__(self):
        super().__init__()
        #self.geometry("420x240")    #Tk window size
        w = 735 # width for the Tk root
        h = 415 # height for the Tk root
        ws = self.winfo_screenwidth() # width of the screen
        hs = self.winfo_screenheight() # height of the screen
        # calculate x and y coordinates for the Tk root window
        x = (ws/2) - w/2
        y = (hs/2) - h/2
        self.geometry("%dx%d+%d+%d" % (w, h, x, y))    # set the dimensions of the screen and where it is placed

        self.configure(bg='whitesmoke')
        #style = ttk.Style(self)
        #style.theme_use('clam')

        self.title("summary file")#Tk window title

```



```

self.process_window()

def process_window(self):

    self.grid_columnconfigure(0, minsize=170)
    self.grid_columnconfigure(2, minsize=170)
    self.grid_columnconfigure(4, minsize=170)
    self.grid_columnconfigure(1, minsize=10)
    self.grid_columnconfigure(3, minsize=10)

    for j in range(15):
        Frame(self, width=20, height=20, bg='whitesmoke').grid(column=1, row=j)

    #Temperature
    self.label_temperature=Label(text="Temperature (°C) :",font=("Helvetica",11), bg='lightgrey')
    self.label_temperature.grid(row=3, column=0, sticky = 'w')

    self.temperature = ttk.Combobox(self, width=15)
    self.temperature['values']=[100,120]
    self.temperature.current(1)
    self.temperature.grid(row=3, column=1, sticky='w')
    self.temperature.configure(state="readonly")

    #Time
    self.label_time=Label(text="Time (min) :",font=("Helvetica",11), bg='lightgrey')
    self.label_time.grid(row=5, column=0, sticky = 'w')

    self.time = ttk.Combobox(self, width=15)
    self.time['values']=[5,10,15,20,25,30]
    self.time.current(0)
    self.time.grid(row=5, column=1, sticky='w')
    self.time.configure(state="readonly")

    #Sample
    self.label_sample=Label(text="Sample :",font=("Helvetica",11), bg='lightgrey')
    self.label_sample.grid(row=7, column=0, sticky = 'w')

    self.sample = ttk.Combobox(self, width=15)
    self.sample['values']=['R1R2','R3','R4','R5R6']
    self.sample.current(0)
    self.sample.grid(row=7, column=1, sticky='w')
    self.sample.configure(state="readonly")

    #lot
    self.label_lot=Label(text="Lot n° :",font=("Helvetica",11), bg='lightgrey')
    self.label_lot.grid(row=9, column=0, sticky = 'w')

    self.lot = ttk.Combobox(self, width=15)
    self.lot['values']=[1,2,3,4,5,6]
    self.lot.current(5)
    self.lot.grid(row=9, column=1, sticky='w')
    self.lot.configure(state="readonly")

    #Bach
    self.label_bach=Label(text="Bach n° :",font=("Helvetica",11), bg='lightgrey')

```

```

self.label_bach.grid(row=3, column=3, sticky = 'w')

self.bach = ttk.Combobox(self, width=15)
self.bach['values']=[1,2,3,4]
self.bach.current(1)
self.bach.grid(row=3, column=4, sticky='w')
self.bach.configure(state="readonly")

#placement
self.label_placement=Label(text="Placement :",font=("Helvetica",11), bg='lightgrey')
self.label_placement.grid(row=5, column=3, sticky = 'w')

self.placement = ttk.Combobox(self, width=15)
self.placement['values']=['Haut','Bas','Gauche','Droite']
self.placement.current(0)
self.placement.grid(row=5, column=4, sticky='w')
self.placement.configure(state="readonly")

#sheet
self.label_sheet=Label(text="sheet n° :",font=("Helvetica",11), bg='lightgrey')
self.label_sheet.grid(row=7, column=3, sticky = 'w')

self.sheet = ttk.Combobox(self, width=15)
self.sheet['values']=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
self.sheet.current(0)
self.sheet.grid(row=7, column=4, sticky='w')
self.sheet.configure(state="readonly")

#ink
self.label_ink=Label(text="ink :",font=("Helvetica",11), bg='lightgrey')
self.label_ink.grid(row=9, column=3, sticky = 'w')

self.ink = ttk.Combobox(self, width=15)
self.ink['values']=['Bectron']
self.ink.current(0)
self.ink.grid(row=9, column=4, sticky='w')
self.ink.configure(state="readonly")

#Button to start the process
self.go_button = Button(text="START", command=self.process, cursor="hand2", bg='blue', fg='lightgrey',
activebackground='darkslateblue', activeforeground='lavender')
self.go_button.grid(row=15, column=2, sticky='ew')

def process(self):
    #get all the values
    gtemp=self.temperature.get()
    gtime=self.time.get()
    gsample=self.sample.get()
    glot=self.lot.get()
    gbach=self.bach.get()
    gplace=self.placement.get()
    gsheet=self.sheet.get()
    gink=self.ink.get()

    df=pd.DataFrame()
    filenumberT71=0
    filenumberT72=0

```

```

print(df)

print(gtemp)
print(gtime)
print(gsample)
print(glot)
print(gbach)

#path to folder
Mainfolder =filedialog.askdirectory(initialdir="E:\DACTEM\LO6B1T120t5")
mfname=os.path.basename(Mainfolder)
date=mfname.rsplit('_')[-2]
hour=mfname.rsplit('_')[-1]

print (date)
print (hour)
steplist=os.listdir(Mainfolder)

steplist.remove("T1.csv")

print ("steplist =",steplist)
interstingl = 1
interstingr = 1
path=[]
datalist=[]
for step in steplist :

    stepcontents=os.listdir(Mainfolder+'/'+step)
    path.append(Mainfolder+'/'+step)
    counter=0
    for filename in stepcontents:
        counter=counter+1
        fname=filename.rsplit('.')[0]
        df.loc[fname,'number']=fname
        print (fname)

    file = Mainfolder+'/'+step+'/'+filename

    if step == "T7_1" :
        filenumberT71=filenumberT71+1
        data=pd.read_csv(file, header=0, decimal=',', sep=';')
        RPdata=data.iloc[interstingl,interstingr]
        df.loc[fname,'RP']=RPdata

    elif step == "T7_2" :
        filenumberT72=filenumberT72+1
        data=pd.read_csv(file, header=0, decimal=',', sep=';')
        RSdata=data.iloc[interstingl,interstingr]
        df.loc[fname,'RS']=RSdata

    if gsample == "R3" :
        if gplace == "Gauche":
            df.loc[fname,'Shape']=dshapelse[str(fname)]
            df.loc[fname,'length']=dlenght3g[str(fname)]
            df.loc[fname,'width']=dwidth123456[str(fname)]

```

```

elif gplace == "Droite" :
    df.loc[fname,'Shape']=dshape3d[str(fname)]
    df.loc[fname,'lenght']=dlenght3d[str(fname)]
    df.loc[fname,'width']=dwidth3d[str(fname)]

df.loc[fname,'Sample']=gsample

elif gsample == "R1R2" and gplace == "Haut":
    df.loc[fname,'Shape']=dshapelse[str(fname)]
    df.loc[fname,'width']=dwidth1256h[str(fname)]
    df.loc[fname,'lenght']=dlenght1256h[str(fname)]
    if int(fname) < 21 :

        df.loc[fname,'Sample']="R2"
    else :
        df.loc[fname,'Sample']="R1"

elif gsample == "R1R2" and gplace == "Bas" :
    df.loc[fname,'Shape']=dshape12456b[str(fname)]
    df.loc[fname,'width']=dwidth123456[str(fname)]
    df.loc[fname,'lenght']=dlenght12456[str(fname)]
    if int(fname) < 21 :
        df.loc[fname,'Sample']="R1"
    else :
        df.loc[fname,'Sample']="R2"

elif gsample == "R5R6" and gplace == "Haut" :
    df.loc[fname,'Shape']=dshapelse[str(fname)]
    df.loc[fname,'width']=dwidth1256h[str(fname)]
    df.loc[fname,'lenght']=dlenght1256h[str(fname)]
    if int(fname) < 21 :
        df.loc[fname,'Sample']="R6"
    else :
        df.loc[fname,'Sample']="R5"

elif gsample == "R5R6" and gplace == "Bas" :
    df.loc[fname,'Shape']=dshape12456b[str(fname)]
    df.loc[fname,'width']=dwidth123456[str(fname)]
    df.loc[fname,'lenght']=dlenght12456[str(fname)]
    if int(fname) < 21 :
        df.loc[fname,'Sample']="R5"
    else :
        df.loc[fname,'Sample']="R6"

elif gsample == 'R4' and gplace== 'Haut' :
    df.loc[fname,'Shape']=dshapelse[str(fname)]

    df.loc[fname,'width']=dwidth4h[str(fname)]
    df.loc[fname,'lenght']=dlenght4h[str(fname)]
    df.loc[fname,'Sample']="R4"

elif gsample == 'R4' and gplace== 'Bas' :
    df.loc[fname,'Shape']=dshape12456b[str(fname)]
    df.loc[fname,'Sample']="R4"
    df.loc[fname,'width']=dwidth123456[str(fname)]

```

```

df.loc[fname,'lenght']=dlenght12456[str(fname)]

else :
    df.loc[fname,'Sample']=gsample
    df.loc[fname,'width']=dwidth123456[str(fname)]
    df.loc[fname,'lenght']=dlenght12456[str(fname)]

df.loc[fname,'Placement']=gplace

df.loc[fname,'Temperature']=gtemp
df.loc[fname,'Time']=gtime
df.loc[fname,'Lot']=glot
df.loc[fname,'Bach']=gbach
df.loc[fname,'Orientation']=dorient[gsample]
df.loc[fname,'Sheet']=gsheet
df.loc[fname,'Ink']=gink

df.loc[fname,'Energy']=float(gtemp)*float(gtime)

print(df)
savepath=filedialog.askdirectory(initialdir="H:\Resume dactem")

df.to_csv(savepath+'/'+L'+glot+'_B'+gbach+'_S'+gsample+'_t'+gtime+'_T'+gtemp+'_Place'+gplace+'_Sheet'+gsheet+'_Ink'+gink+'_summary_'+date+'_'+hour+'.csv', index=False, sep=',', decimal='.') #Save the dataframe to a csv file. If encoding problem in the future : https://stackoverflow.com/questions/57061645/why-is-%C3%82-printed-in-front-of-%C2%B1-when-code-is-run

#define the creation of the csv

if __name__ == '__main__':
    window = GUI()
    window.mainloop()

```

Programme de concaténation morphologique

```

import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
plt.style.use('ggplot')

import pandas as pd
import os
import seaborn as sns
from pprint import pprint

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from statistics import mean
import cmath, math
import numpy as np
import random
import csv

import tkinter as tk
from tkinter import *
from tkinter import ttk

```

```

from tkinter import filedialog
import tkinter.scrolledtext as st

import sys
import re

import scipy.optimize
from scipy.optimize import curve_fit
from scipy.fft import fft, fftfreq

from matplotlib.ticker import AutoMinorLocator

pd.options.mode.chained_assignment = None

from pathlib import Path

dic_name = {1 : 'L0.15l5R1', 2 : 'L0.25l5R1', 3 : 'L0.5l5R1', 4 : 'L1l5R1', 5 : 'L2l5R1', 6 : 'L0.15l10R1', 7 : 'L0.25l10R1', 8 :
'L0.5l10R1', 9 : 'L1l10R1', 10 : 'L2l10R1', 11 : 'L0.15l15R1', 12 : 'L0.25l15R1', 13 : 'L0.5l15R1', 14 : 'L1l15R1', 15 : 'L2l15R1',
16 : 'L0.15l5R2', 17 : 'L0.25l5R2', 18 : 'L0.5l5R2', 19 : 'L1l5R2', 20 : 'L2l5R2', 21 : 'L0.15l10R2', 22 : 'L0.25l10R2', 23 :
'L0.5l10R2', 24 : 'L1l10R2', 25 : 'L2l10R2', 26 : 'L0.15l15R2', 27 : 'L0.25l15R2', 28 : 'L0.5l15R2', 29 : 'L1l15R2', 30 : 'L2l15R2',
31 : 'L2l2.5R6', 32 : 'L1l2.5R6', 33 : 'L0.5l2.5R6', 34 : 'L0.25l2.5R6', 35 : 'L0.15l2.5R6', 36 : 'L2l1R6', 37 : 'L1l1R6', 38 :
'L0.5l1R6', 39 : 'L0.25l1R6', 40 : 'L0.15l1R6', 41 : 'L2l2.5R5', 42 : 'L1l2.5R5', 43 : 'L0.5l2.5R5', 44 : 'L0.25l2.5R5', 45 :
'L0.15l2.5R5', 46 : 'L2l1R5', 47 : 'L1l1R5', 48 : 'L0.5l1R5', 49 : 'L0.25l1R5', 50 : 'L0.15l1R5'}

#GUI
class GUI(Tk):
    def __init__(self):
        super().__init__()
        #self.geometry("420x240")      #Tk window size
        w = 400 # width for the Tk root
        h = 200 # height for the Tk root
        ws = self.winfo_screenwidth() # width of the screen
        hs = self.winfo_screenheight() # height of the screen
        # calculate x and y coordinates for the Tk root window
        x = (ws/2) - w/2
        y = (hs/2) - h/2
        self.geometry('%dx%d+%d+%d' % (w, h, x, y))      # set the dimensions of the screen and where it is placed

        self.configure(bg='whitesmoke')
        #style = ttk.Style(self)
        #style.theme_use('clam')

        self.title("summary file")#Tk window title

        self.process_window()

    def process_window(self):

        self.grid_columnconfigure(0, minsize=400)

        for j in range(9):
            Frame(self, width=400, height=20, bg='blue').grid(column=1, row=j)

        #sheet number

```

```

self.label_sheet=Label(text="sheet n° :",font=("Helvetica",11), bg='lightgrey')
self.label_sheet.grid(row=1, column=0)

self.sheet = ttk.Combobox(self, width=15)
self.sheet['values']=['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10','S11','S12','S13','S14','S15']
self.sheet.current(0)
self.sheet.grid(row=2, column=0)
self.sheet.configure(state="readonly")

#Bach number
self.label_batch=Label(text="Batch n° :",font=("Helvetica",11), bg='lightgrey')
self.label_batch.grid(row=3, column=0)

self.batch = ttk.Combobox(self, width=15)
self.batch['values']=['B1', 'B2', 'B3', 'B4', ]
self.batch.current(0)
self.batch.grid(row=4, column=0)
self.batch.configure(state="readonly")

#button to concat files
self.go_button = Button(text="START", command=self.process, cursor="hand2", bg='blue', fg='lightgrey',
activebackground='darkslateblue', activeforeground='lavender')
self.go_button.grid(row=8, column=0)

def process(self):
    fig = plt.figure()
    ax1 = fig.add_subplot(111)

    gsheet=self.sheet.get()
    gbatch=self.batch.get()
    section_df = pd.DataFrame()
    counter = 0

    try :
        allfiles=filedialog.askopenfilename(initialdir = "H:/profilo", filetypes=(("csv files" , "*.csv"),), title="Select one or more
files", multiple=True)
        print ('-----')
        print ('-----')
    except UnicodeDecodeError :
        print("Error : please choose a data file")
    if not allfiles: #If the operator didn't choose a file
        print("No files selected")

    foldername=os.path.dirname(allfiles[0])
    colorlist=sns.color_palette("hls", len (allfiles))
    for file in allfiles :

        counter= counter+1

        if file.endswith('.txt') :
            p = Path(file)
            p.rename(p.with_suffix('.csv'))
            file=file.rsplit('.')[0]+''.csv'

    colorlist=sns.color_palette("hls", len (allfiles))

```

```

file=file.replace("\\','")
filename=file.replace(foldername,"")
filename=filename[1:]
filename=filename.rsplit('.')[0]

#dictionary
nfile = int (re.sub("[^0-9]", "", filename))
if nfile >= 51 :
    nfile = nfile-50

merged_df=pd.read_csv(file, header=7, sep='t', decimal='.')#read resume file

half_index=len(merged_df.index)/2
df1 = merged_df.iloc[:int(half_index)]
df2 = merged_df.iloc[int(half_index):]

min_df1=df1[' Wavi'].min()
i_min_df1 = df1[[' Wavi']].idxmin()

min_df2=df2[' Wavi'].min()
i_min_df2 = df2[[' Wavi']].idxmin()

df1[' Wavi'] = df1[' Wavi']-min_df1
df2[' Wavi'] = df2[' Wavi']-min_df2

df1 = df1.iloc[int(i_min_df1):]

df2=df2.iloc[:int(i_min_df2)-int(half_index))]

merged_df=pd.concat([df1,df2])

Section = merged_df[' Wavi'].sum()*10**(-16)
#print ('Section of ',filename, ' : ', Section)

section_df.loc[counter,'Parameters']=dic_name[nfile]
section_df.loc[counter,'Section']=Section
ax1.plot(merged_df['Unnamed: 0'], merged_df[' Wavi'], color=colorlist[counter-1])

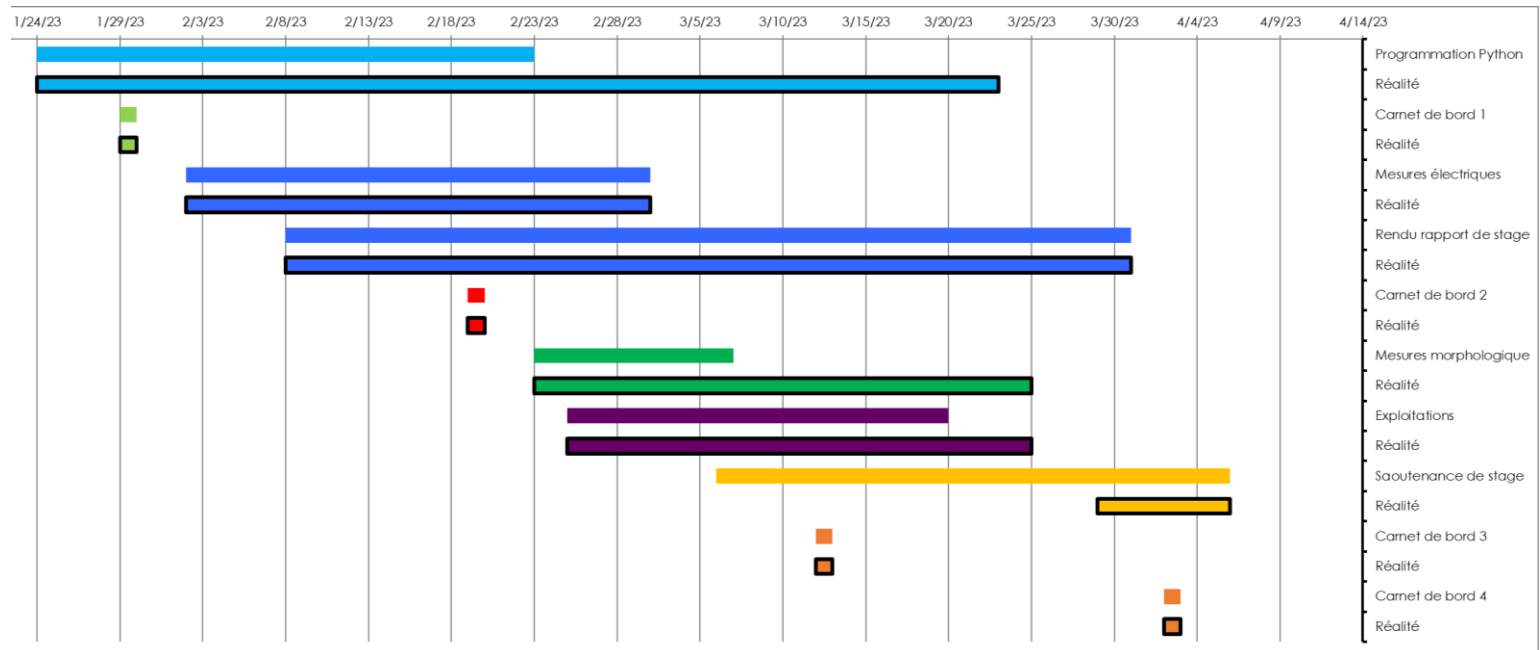
section_df=section_df.groupby('Parameters').mean().reset_index()
with pd.option_context('display.max_rows', None, 'display.max_columns', None): # more options can be specified also
    print(section_df)
print (len(section_df), ' file compiled')

savepath=filedialog.askdirectory(initialdir="H:\Profilo\Section file", title="Select a saving path")
section_df.to_csv(savepath+'/'+gsheet+gbatch+'.csv', index=False, sep=',', decimal='.') #Save the dataframe to a csv
file. If encoding problem in the future : https://stackoverflow.com/questions/57061645/why-is-%C3%82-printed-in-front-of-%C2%B1-when-code-is-run
print ('Data compiled successffuly.')
plt.show()

if __name__ == '__main__':
    window = GUI()
    window.mainloop()

```


ANNEXE D : Planning de stage



**FIGURE 31 : PLANNING PREVISIONNEL ET REEL
DU DEROULEMENT DU STAGE**

Abstract

This study concerns the potential of printing conductive ink on flexible substrates as an alternative to "Printed Circuit Boards" (PCBs) in electronic circuits. The advantages include low cost, flexibility, and recyclability, however, limitations in circuit design and ink quality must be considered.

Therefore, the goal of this study is to establish an automatic electrical and morphological characterization protocol for conductive inks printed by screen printing.

In order to establish this protocol, it is necessary to measure the resistivity through resistance and section. An automatic programme has been developed to measure the section with a profilometer and a recipe was created to measure the resistance of tracks. In order to treat data, two codes were developed to extract and make a summary of section and resistance data, finally a third code will create a PDF file using the data summarized to characterize the printed ink.

The results of this study show a decreasing trend of the resistivity with the growth of energy brought during the thermal annealing. Due to the limit of screen-printing, the calculated resistivity was not constant and shows big variation for annealing conditions.

In conclusion, 3 programmes have been developed in order to characterize screen-printed inks efficiently. Nevertheless, the mask used to print the ink may have to be re-designed to measure a resistivity with more accuracy.

Keywords:

Screen printing, Printed inks, Resistivity, Section, Code, Characterization, Thermal annealing